Did the Action Already Happen? Reliable Recovery for AI Agents Under Ambiguous Tool Outcomes

Ankitha Vuyyala
Independent Researcher, Louisville, Kentucky, USA
ankithavuyyala08@gmail.com
ORCID: https://orcid.org/0009-0004-1916-8070

October 2026

ABSTRACT

When a tool call made by an AI agent times out or loses its reply, the agent cannot tell whether the effect happened, did not happen, or is still on its way. The standard remedy is an idempotency key, but in a documentation census of 52 widely used write APIs, 38 offer no server-side key on the operation an agent would call, and several keys that exist expire within hours. I argue that recovery needs *attributability*, the ability to count the standing effects of one's own operation, and derive for each API contract and read model whether a client can achieve prevention of duplicates, convergence of the final state, or only honest abstention; any policy that must stop at a horizon has to bet between two indistinguishable worlds. Of the 38 key-less APIs, 31 persist a client-controlled field that can be looked up. I build Settle, a recovery layer that journals each intent, stamps an operation id into such a field, resolves unknown outcomes by that id up to a horizon, and compensates only effects it can prove are surplus. Under internally frozen protocols I evaluate it on six unmodified services in containers, with injected network faults and process crashes, against ten baselines including idempotency keys, a Temporal workflow engine and a client-side guard, with a scripted agent and four language models. Across the attributable and reversible grid, Settle reduces inconsistent final states from 34.7% for the strongest matched-latency baseline of the main study to 3.2%; without the two fault modes that structurally require a final sweep or row-level checks, the rates are 29.4% and 3.4%. Durable keys are exact only where the service honours them. A follow-up study of 4,272 episodes bounds the claims: a tag-based wait-then-count policy matches Settle on every ordinary fault (it is inconsistent in 10.8% of these attributable and reversible worlds against Settle's 3.2%, with 42 of its 65 failures on transport redelivery), so convergence pays off mainly for duplicates the transport creates; an author filter combined with waiting rescues content checks against look-alikes from other accounts; and a frontier model without the layer fails in the same cells as smaller ones (52.2% of hard worlds, against 6.0% with Settle).

Keywords: LLM agents; tool use; exactly-once; idempotency; fault tolerance; compensation; durable execution; API design

1. Introduction

An agent that acts on the world through tools inherits the oldest problem of distributed computing. It sent a request and no answer came back. The request may have been lost on the way; it may have been carried out with only the reply lost; or it may still be in flight and land a few seconds after the agent has given up. The three cases look identical to the agent (Fig. 1). If the agent retries, it creates a second issue, a second e-mail or a second payment whenever the first attempt went through. If it does not, the task stays incomplete whenever the first attempt did not.

The problem is as old as the two-generals argument [1], [2], and it has a standard engineering answer: give each logical operation a key and let the server execute each key at most once [3]–[5]. Recent work shows that language-model agents run into exactly this problem and that the standard answer does not carry over by itself. Models frequently re-issue a write after an ambiguous failure without first checking whether it took effect [6], [7]. Li [8] proves that no policy which only observes outcomes can be exactly-once while a request may still be in flight, and finds that idempotency keys remove most duplicates, but only on tools that honour them. Trofimov and Novikov [9] argue that without a primitive that yields one authoritative outcome, no recovery protocol is both safe under unknown outcomes and safe to compensate. Agent runtimes that add transactions [10], [11], durable execution [12], [13] or rollback [14], [15] reach

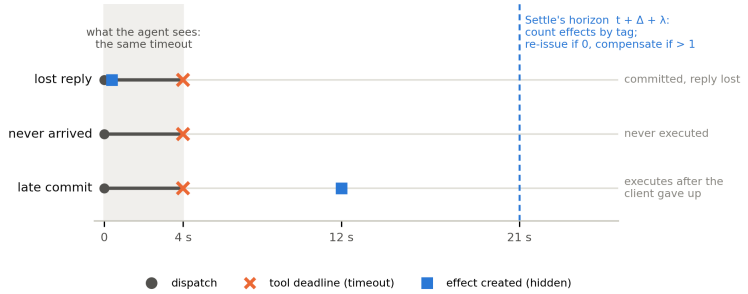


Fig. 1. Three worlds that give the agent the same observation. A request whose reply was lost, one that never arrived, and one that will commit after the agent has given up all end in the same timeout. Settle decides only after the in-flight bound Δ and the read lag λ have passed, by counting the effects that carry the operation's tag.

exactly-once behaviour only when the downstream service honours a key or when failures are detectable, and neither holds for a lost reply.

How large is the gap that this leaves? I examined the documentation of 52 widely used write APIs across payments, developer tools, messaging and data services (Section 5). Only 14 of them offer a server-side key on the operation an agent would call, and most of those are payment APIs. Of these 14 keys, 4 are documented to expire within six hours or when the object's state changes. None of the twelve issue trackers and project tools and none of the thirteen messaging and mail services I examined offers one. These are services where many agent writes land and where a duplicate is visible to people. For these services, recovery without a key is not a corner case but the common case.

The key observation. Preventing a duplicate requires a key, but *repairing* one requires only that the client can tell which effects belong to which of its operations. Many key-less APIs persist a field that the client controls: an issue label or body, a mail header, a metadata map, a reference column, a custom property. If the client writes an operation id into that field, it can later count the effects of its own operation exactly. A search by content cannot do that, because other users may create records with the same content. With an exact count, the client can wait out the in-flight window, re-issue only if no effect exists, and remove any surplus copy it can prove is its own. In the census, 31 of the 38 key-less APIs offer such a field together with a way to look it up, and 21 of them offer an exact lookup. Payment engineers know this "client reference plus authoritative lookup" pattern informally. To my knowledge it has not been formalised for agents, compared against keys and workflow engines, or tested in the places where it must fail.

This paper makes four contributions.

1. **A theory of what recovery can achieve** (Section 3, Section 4). I define *attributability* as the identifiability of the number of standing effects of an operation from what the client can observe. The notion unifies native keys, client tags, writer exclusivity and returned ids. I prove which guarantee each contract and read model admits. Without a known in-flight bound, prevention requires a durable key, and a key protects only within its retention. Convergence of the final state is achievable for attributable, reversible effects without any key; for irreversible effects it collapses to prevention. Content-based verification is unsound once look-alikes exist, aggregates are in general attributable only under exclusive writing, and the tag must survive a crash between dispatch and record. Finally, a policy that must stop at a horizon faces two indistinguishable worlds, so its residual risk is bounded below by the less likely of them. Settle's rule attains that bound when, given what it has seen, a late landing is less likely than a lost request. The result is an achievability map that predicts, cell by cell, where each recovery method must fail.
2. **A census of real API contracts** (Section 5). I code 52 APIs from their official documentation and re-check a random sample in a second pass. The census places real services in the achievability map and measures the premise of the paper rather than asserting it.
3. **Settle, a recovery layer** that implements the constructive side of the theory (Section 6). It writes each intent to a durable journal before dispatch, stamps an operation id into a persisted field, resolves unknown outcomes by that id after the read path's lag, and keeps reconciling up to a horizon. It compensates only effects it can prove are surplus, including effects the agent itself asks to delete, and reports *unknown* or an unrecoverable duplicate where nothing can be guaranteed.
4. **An internally pre-registered evaluation on real services with real faults** (Section 7, Section 8). Six services run in containers: PostgreSQL, Gitea, Mailpit, Redis, SeaweedFS and TigerBeetle. Faults are injected at the

network and process level and validated against each service's own records: lost replies, requests that never arrive, held connections, late commits with fixed and heavy-tailed delays, partial batches, transport redelivery and agent-process crashes. I compare sixteen conditions, namely ten baselines, Settle, four ablations of Settle and an outcome oracle, under a scripted agent and three OpenAI models. Grading uses service-side records only, and hypotheses, analysis code and source hashes were frozen internally before the first test episode. A second, separately frozen follow-up study of 4,272 episodes adds the strongest tag-based alternative I could build, Settle without its implementation gaps, delayed redelivery, look-alikes from another account, plan-free intent binding and a frontier model (Section 9).

The evidence locates the boundary that the theory predicts. The scripted grid replays 1,188 worlds under fifteen conditions each (17,820 episodes), and the language-model grid adds 2,680 episodes. In ambiguous worlds whose write is attributable and reversible, Settle leaves 3.2% of episodes inconsistent, against 34.7% for the strongest baseline, a guard that waits exactly as long as Settle does. Without the two fault modes that every baseline fails by construction, redelivery and partial batches, the rates are 3.4% and 29.4%. Every remaining Settle failure is accounted for by the theory: either Settle's assumed in-flight bound was violated, whose consequence the theory bounds and which trades against latency, or the write lies in a cell where the theory admits no client-side guarantee. Durable keys, here those of a workflow engine, are exact where the service honours the key, but on key-less writes they are no better than plain retries (61.0% for the workflow engine against 61.0% for plain retries). Verification by content, the basis of current client-side guards, fails in more than half of the worlds that contain same-account look-alike records. Language-model agents either re-issue blindly or check first. Checking helps, but it fails under look-alikes, after crash replay and under long delays: with my worst-case look-alikes and under long delays the observation itself is insufficient, and after a crash the framework re-issues before the model can check. With Settle underneath, all three models fall to 6.0% inconsistency in these hard worlds. Where nothing can be guaranteed, as with shared aggregates and redelivered e-mail, every method short of an outcome oracle fails, and Settle reports these cases instead of claiming success. The follow-up study draws the boundaries more tightly, including where they work against Settle. A tag-based wait-then-count policy is exactly as good as Settle on ordinary faults, so Settle's convergence pays off mainly for duplicates created by the transport. A content check with an author filter, combined with waiting out the bound, is as good as a tag when look-alikes come from another account. And the frontier model `gpt-5.6-sol` without a recovery layer leaves 52.2% of hard worlds inconsistent, failing in the same cells as the smaller models, against 6.0% with Settle.

Section 2 positions the work. Section 3 and Section 4 give the model and the propositions, Section 5 presents the census, and Section 6 describes Settle. Section 7 and Section 8 present the experimental design and the results, and Section 9 the internally pre-registered follow-up study. Section 10 draws the consequences for API and agent-framework design and states the limitations.

2. Background and related work

Exactly-once effects in distributed systems. Two parties cannot agree on whether a message was acted upon when messages can be lost [1], [2]. Practical systems therefore build *effectively-once* behaviour from at-least-once delivery plus deduplication. The ingredients are idempotent operations and request identifiers [3], durable completion records that let a server recognise a retried request [4], transactional logs in stream processors [16], and exactly-once semantics for stateful serverless workflows [17]. Commercial APIs expose the same idea as an idempotency key [5], [18]. Durable-execution engines persist workflow progress but execute each activity at least once, and advise that activities be idempotent [12], [13]. When an operation cannot be undone atomically, sagas and compensating transactions restore consistency by running a semantic inverse [19], [20]. All of these mechanisms assume either that the server deduplicates or that the client knows which effects exist. My setting removes both assumptions and asks what remains. The answer, attribution through a reference the client writes, is close to what payment integrations do with merchant references. My contribution is to state when it suffices and when it cannot, and to measure how it compares with keys and durable execution when an autonomous agent drives the calls.

Ambiguous tool outcomes in language-model agents. Sun [6] builds observation-equivalent pairs of worlds, in which a write either did or did not execute, and shows that an agent without further information can be right in at most half of them. IdempotencyBench [7] measures idempotency violations under retries with a hidden effect ledger. Li [8] gives the most complete study to date: twelve fault modes on simulated services, nine models and three production harnesses. That study proves that no verification-only policy is exactly-once under late commits without an in-flight bound. It also finds that idempotency keys remove most duplicates, and that a client-side guard helps weaker models more than

frontier models. Trofimov and Novikov [9] model effect histories and name the anomalies that arise at the agent-tool boundary: duplicated, missing, orphaned compensation and residue. They argue that without an authoritative-outcome primitive no protocol is both safe under unknown outcomes and safe to compensate. Their census of tool annotations in the Model Context Protocol shows that such annotations cannot express these properties, and the specification itself calls its idempotency and destructiveness annotations untrusted hints [21]. Wang [22] shows that interfaces which expose effect semantics reduce duplicate effects.

My work accepts Li's impossibility result for *prevention* and studies *convergence* instead. It also differs from this line of work in four respects:

- I *construct* the missing outcome primitive from fields that ordinary APIs already persist, and measure in a census how often such fields exist; Trofimov and Novikov's census covers tool annotations, not the services behind the tools.
- I evaluate on real services rather than simulated ones.
- I add agent-process crashes, a workflow engine, compensation-based baselines, look-alike records from concurrent users and aggregate-only reads.
- I prove a lower bound for policies that must stop at a horizon, which characterises the residual failures I measure.

Recovery mechanisms for agents.

- *Verification.* Verify-before-retry wrappers check a postcondition before repeating a call [23]. Li's guard adds automatic keys, a re-check after the documented lag and a block on repeats it cannot verify [8].
- *Transactions.* Atomix gates commits on progress, compensates failed branches and deduplicates ambiguous retries with keys [10]. Cordon offers semantic transactions with an effect outbox [11]. Agentic Transaction proposes ACID-style guarantees for data agents [24].
- *Compensation.* Robust Agent Compensation and SagaLLM bring log-based compensation and sagas to agents [14], [15].
- *Checkpoint and restore.* AgentRewind [25] and DART [26] restore agent state. Rolling back agent state does not undo effects in the world, and a restored agent may re-synthesise slightly different requests that a server treats as new [27]. CapLease measures such equivalent re-proposals and binds them to a durable authorisation state [28].

Except for Li's guard and verify-before-retry, these mechanisms assume keyed downstream services or detectable failures. I include baselines from the verification family (verify-before-retry and Li's guard), the compensation family (log-based rollback) and durable execution (a Temporal workflow engine), together with idempotency keys, all with matched retry budgets; transactional runtimes and checkpoint-and-restore are not represented. I also add a variant of the guard that waits as long as Settle, so that waiting alone cannot explain a difference.

Fault injection and reliability benchmarks for agents. Several benchmarks stress agents with tool failures, timeouts, rate limits or corrupted outputs [29]–[32]. Related work studies guards under state races [33], revisable streaming execution [34], and agents that claim success when the state disagrees [35]. These works measure task success under faults. I measure the correctness of the *final external state* after an ambiguous write, which task success does not capture, and I also score whether the agent's report is honest about that state.

3. Problem setting

Operations, attempts and effects. A *logical operation* ℓ is one intended change to an external service: create one issue, send one e-mail, make one transfer. The client makes *attempts* $a_1, a_2, \ldots$ of ℓ, dispatched at times $t_1 < t_2 < \ldots$. An attempt either *executes* at an unknown time $t_i + \delta_i$ with $\delta_i \geq 0$, or never executes; the transport may also *redeliver* an attempt, so that it executes a second time although the client issued it once. An executed attempt creates one *effect* e, which is *standing* until it is neutralised. For every attempt the client receives an acknowledgement, an error, a timeout or nothing, and it may issue reads through the read paths that the service offers. The client can crash, and only what it wrote to durable storage survives a crash. Other actors share the service and create effects of their own.

A *world* w fixes, for every attempt, whether, when and how often it executes, together with the other actors' effects. Worlds are drawn from a prior that describes the deployment, such as how often requests are lost, how delays are distributed and how active other users are. The client's *history* h is the sequence of its own actions, the responses and the read results, and $\Pr[\cdot \mid h]$ denotes the posterior over worlds. A *policy* maps histories to actions: re-issue, read, wait, neutralise an effect, or report. Two worlds are *observation-equivalent up to time* t if every policy receives the same history in both until t.

Contract properties of a write. Three properties belong to the service's interface, not to the agent. The first is a *native key* (class K): attempts that carry the same key create at most one effect, and a repeat returns the original outcome or an "exists" refusal. Real keys come with a *retention* r, after which the service forgets the key and treats a repeat as new. The second is the *read model*, which says what reads reveal about standing effects. Under an *exact* read model, reads return all standing effects with their fields. A *lagged* read model is exact, but only for effects older than a lag λ. Under a *content* read model, effects are found by their content, and other actors may create effects with identical content, which I call *look-alikes*. An *aggregate* read model returns only a count or a sum. The third property is *reversibility*. A *targeted* compensation $c(e)$ neutralises exactly the effect e, such as deleting issue #n or row r. A *fungible* compensation neutralises one unit of an aggregate, such as a decrement. Some effects cannot be neutralised at all; a delivered e-mail is the standard example.

In-flight bound. An attempt that has not executed within Δ of its dispatch never executes. Δ may be unknown to the client, or the delay distribution F may have a heavy tail, in which case any finite assumed bound is sometimes wrong.

Attributability. The central notion is the following.

> **Definition 1 (Attributability).** Under a read model with lag λ, operation ℓ is *attributable* if, for every time t, the number $N_\ell(t-\lambda)$ of standing effects of ℓ that executed before $t-\lambda$ is a function of the client's history at t. Equivalently, no two worlds that are observation-equivalent up to t disagree on $N_\ell(t-\lambda)$.

Attributability concerns effects that already exist. It does not make the *eventual* count identifiable, because an attempt may still be in flight. That additionally requires the in-flight bound: once every attempt is older than Δ, the eventual count equals the attributable count. The theory below separates the two on purpose, because attribution is a property of the API, whereas the in-flight bound is a property of the network and the service's queues.

Five mechanisms make an operation attributable (Table 1). A native key bounds N_ℓ by one and answers queries for it. A *client tag*, an operation id τ written into a field that the service persists and returns, lets a read count exactly the effects that carry τ, provided tags are unique per operation and no one else writes τ into an effect. *Exclusivity*, being the only writer to an aggregate, makes the change in the aggregate equal to N_ℓ. If no one else writes with the client's account and the service records authorship, the account itself identifies the client's effects, as the follow-up study measures (Section 9). Finally, an id returned by the service identifies the effect, but it is lost exactly when the reply is lost, which is the case that matters.

Table 1. Five ways an operation becomes attributable. The returned id, the most common API design, gives no attributability in the ambiguous case.

Source	Why N_ℓ becomes identifiable	Example in my stack
Native key	the service enforces $N_\ell \leq 1$ and answers "exists"	TigerBeetle transfer id; S3 conditional create
Client tag	every effect carries τ; a read returns all effects with τ	Gitea issue body, mail Message-ID, row column, stream field
Exclusivity	only the client writes, so the aggregate's change equals N_ℓ	a counter with no other writer
Own account	only the client writes with its account, and the service records the author	Gitea issues and comments when other users use other accounts
Returned id	the reply names the new effect	lost together with the reply

Guarantees. I distinguish three guarantees. *Prevention* means that ℓ executes exactly once over the episode; it is the target of keys and of prior agent benchmarks [8]. *Convergence at horizon H* means that from time H on exactly one effect of ℓ stands, including after every in-flight attempt has landed, and that no effect of another operation or actor has been neutralised; a transient duplicate is allowed if it is removed by H. *Safe abstention* means that, where neither guarantee is achievable, the client reports ℓ as unknown and never claims completion; it is a property of the report, not of the state. Prevention implies convergence once the single execution has landed, provided no other actor's effect was touched; convergence does not imply prevention. My thesis is that convergence is the right target wherever prevention is provably impossible and effects are attributable and reversible.

Primary metric. The primary outcome of every episode is therefore *residual inconsistency at quiescence*. After the client finishes and every in-flight request has landed, an episode is inconsistent if a required effect is missing, if a surplus effect stands, or if another actor's effect was removed. I also report duplicate executions including compensated ones,

which is the prevention metric of prior work; irreversible duplicates; missing effects; collateral damage; and *overclaim*, an inconsistent final state with nothing reported as uncertain.

4. What recovery can achieve

I now state which guarantee is achievable under each contract and read model. Proofs are in Appendix A. Several of the arguments are short indistinguishability arguments. Their value lies less in any single proposition than in the map they add up to (Table 2), which says, for every combination of contract, read model and fault, which guarantee some client-side policy can achieve. Propositions 1–6 and the predictions of the map that the hypotheses test, including where Settle itself must fail, were frozen with the analysis code before any test episode ran. The retention clause of Proposition 1, the corrected statements of Propositions 2, 3, 5 and 6, and Proposition 7 were worked out while writing, after the runs, and I use them to interpret the results rather than as tested predictions. Relative to the frozen map, Table 2 changes the prevention column in two rows (prevention by waiting is possible for reversible and exclusively written aggregate writes when Δ is known) and the wording of three others; the convergence predictions that the hypotheses test are unchanged. The frozen version is in the released repository.

Proposition 1 (without a known in-flight bound, prevention needs a durable key, within its retention). Suppose the service deduplicates a key for a retention period r after it first records it. Re-issuing with the same key then prevents duplicates if the key is a function of *durable* state and every execution of every attempt falls within r of the first execution. If the number of attempts is capped, this gives at most one execution, and exactly one only if some attempt executes. With a known in-flight bound, a sufficient condition is $t_{\text{last}} - t_1 + \Delta < r$. If the key is regenerated after a client crash, as an in-memory random key would be, two attempts with different keys can both execute, and prevention fails. If an attempt executes later than r after the first execution, the service no longer recognises the key, and the operation is protected only by whatever attribution the contract offers apart from the key. Retention that ends with a state change rather than after a time, such as a deduplication key that lasts while an alert is open, behaves in the same way, with the state change in place of r. Without a key, no policy that acts only on observations prevents duplicates under late commits when Δ is unknown [8].

The retention condition matters in practice. A key that expires after one hour does not protect an agent that resumes after a crash the next morning, or one that waits for a human approval before retrying. The census (Section 5) finds documented retentions from one hour to at least thirty days, alongside conditional creates whose protection lasts as long as the object exists.

Proposition 2 (convergence for attributable, reversible writes). Assume the following:

(a) the tag τ of ℓ can be recovered after a crash, for example from an intent record written before the first dispatch;
(b) every executed attempt persists τ unchanged;
(c) the read path is complete and sound for τ after lag λ;
(d) the in-flight bound Δ holds;
(e) a targeted compensation exists, and it is idempotent or itself keyed or tagged, so that it can be retried through the faulty channel;
(f) no other actor neutralises effects that carry τ;
(g) tags are unique per operation, and no other actor writes τ into an effect.

Consider the following policy, which proceeds in rounds. In each round it waits until $t_{\text{last}} + \Delta + \lambda$ and counts N, the effects that carry τ. If $N = 0$, it re-issues with the same τ and starts a new round. If $N > 1$, it compensates $N - 1$ effects, chosen by id. The state is then consistent from the end of the final round on. The number of rounds is finite with probability one if each attempt executes with positive probability, and a policy that caps the number of rounds, as Settle does, reports *unknown* when the cap is reached. If (d) is violated and delays follow a distribution F, the round that ends at horizon H leaves an inconsistent state only if some attempt executes after $H - \lambda$. If attempts execute independently of each other, that happens with probability at most $\sum_i p_i \left(1 - F(H - \lambda - t_i)\right)$, summed over the attempts i dispatched at t_i, where p_i is the probability that attempt i executes at all; this is a statement about the round as a whole, not conditional on what the round observed. The policy needs no assumption about the agent or about the order of faults.

Proposition 3 (irreversible writes: convergence collapses to prevention). If an effect cannot be neutralised, convergence requires that ℓ never executed twice, so convergence equals prevention, and Proposition 1 applies. With a known Δ, the policy "wait $\Delta + \lambda$, count by τ, re-issue only if $N = 0$" prevents every duplicate that the *client* could

cause. The tag is what makes the test $N = 0$ exact; Proposition 4 shows that a test by content is not exact. No client policy, however, prevents a duplicate that the *transport* creates by redelivering a request the client issued once. For irreversible writes without a key, a redelivered request is therefore an unrecoverable duplicate, and the best a client can do is to detect it by its tag and report it. The same wait-then-count policy would also prevent duplicates of reversible writes. Settle does not use it for them, because convergence lets it replace the wait by compensation.

Proposition 4 (content verification is unsound under look-alikes). Consider a world w_1 in which the client's attempt executed and no look-alike exists, and a world w_2 in which the client's attempt did not execute and another actor created an effect with identical content. Under a content read model the two worlds are observation-equivalent, while N_ℓ differs. Every content-only policy is therefore wrong in one of them. A policy that skips the write leaves ℓ missing in w_2. A policy that re-issues duplicates ℓ in w_1. And a policy that re-issues and then removes one of the two matching records either removes the other actor's effect in w_2 or removes its own new effect, which leaves ℓ missing in w_2. A client tag restores attributability, because look-alikes do not carry τ. Fields that real read models often expose, such as the author account and the creation time, separate the two worlds only when the other actor uses a different account or writes at a distinguishable time; they are a partial substitute for a tag, not a guarantee. The follow-up study (Section 9) measures both cases.

Proposition 5 (aggregates: exclusivity is the only general route). Under an aggregate read model, ℓ is attributable if no other actor writes to the aggregate during the ambiguity window. If other actors may write the same unit, or units of unknown size, then a world in which the client's attempt executed and theirs did not gives the same aggregate as a world in which theirs executed and the client's did not. Neither prevention nor convergence is then achievable without a key, and fungible compensation does not help, because the number of surplus units is not identifiable. The only correct behaviour is safe abstention. (Concurrent writes of known and distinct sizes would keep the count identifiable. My shared-aggregate worlds, like most real counters, do not have that property.)

Proposition 6 (the tag must survive a crash). Suppose the client can crash after dispatching an attempt and before recording its outcome. After the restart, the client must issue ℓ if it was never dispatched and must not issue it blindly if it was. If the persisted state does not show whether ℓ may have been dispatched, and the tag of a possible dispatch cannot be recovered, the two cases are observation-equivalent to the client, which then either duplicates or omits ℓ. Two kinds of durable state suffice to decide correctly: a write-ahead intent record that carries the tag, or a tag derived deterministically from durable step identity, provided every unfinished step is treated after a restart as possibly dispatched and resolved by waiting and counting. A durable step checkpoint *without* a recoverable tag, which is what my drivers keep and what the S−j ablation is left with, is not enough: it shows that the step may have been dispatched, but leaves the client nothing to count by except content, which Proposition 4 shows to be unsound. Workflow engines persist the *scheduling* of an activity durably but execute the activity at least once [12]. Their durable identity is therefore available, but unless it is written into the effect it does not make the effect attributable: durability without attribution yields at-least-once effects. Likewise, an agent framework that replays a pending tool call after a restart turns every crash during a write into a blind re-issue.

Proposition 7 (a policy that stops must bet). Let ℓ be attributable, and let t_d be the time of the policy's last action on ℓ, after which it neither observes nor acts on ℓ. For a reversible write t_d is the final reconciliation; for an irreversible write it is the decision whether to re-issue. Suppose an attempt dispatched at t_0 is still unresolved at t_d. Let w_0 be the worlds in which it never executes, and w_L those in which it executes after $t_d - \lambda$, too late to be seen by the last read. Up to t_d the two kinds of world are observation-equivalent. At t_d, the policy decides how many of its *other* effects of ℓ remain standing: none, by not re-issuing or by compensating its re-issue, or one, by re-issuing or keeping the re-issue. The final count is that number plus one in w_L, so every choice is wrong in either w_0 or w_L. Hence, for every policy π and every history h that satisfies the premise,

$$\Pr[\pi \text{ inconsistent} \mid h] \;\geq\; \min\{\Pr[w_0 \mid h],\ \Pr[w_L \mid h]\}.$$

Settle always keeps one effect standing, so its error at such a history is $\Pr[w_L \mid h]$, plus $\Pr[w_0 \mid h]$ times the probability that its own re-issue fails to execute. It attains the bound exactly when its re-issue executes and a late landing is no more likely, given the history, than an attempt that never executes.

Proposition 7 makes two points about the residual failures I measure. First, they are the price of a bet that every stopping policy must make, not a defect of Settle's rule: a policy that bet the other way would fail in the worlds where the request was lost instead. Settle bets on the lost request because then the operation is complete and the error, if any, is a surplus effect rather than a missing one. In a deployment where late commits after the horizon are more

likely than lost requests, the opposite bet has the lower error. My late-tail worlds are designed so that every request eventually commits, so there the opposite bet would have been right. Second, at a fixed decision time no rule does better than the bound, so the residual risk falls only if t_d moves later or if the posterior shifts. For an irreversible write, a later t_d means a longer wait before the re-issue, which the agent experiences as latency. For a reversible write the reconciliation can instead continue *after* the agent has reported, because compensation needs no agent. Section 8.6 measures the trade-off.

Table 2. Achievability map. For each contract and read model: whether *some* client-side policy can prevent duplicates or make the final state converge, and the policy that attains it. Class K has a native key, class A is key-less but taggable, class U exposes only an aggregate, and class C is a key-less write that can be found only by content. The experiments fill each row with measured rates (Table 7).

Contract	Read model	Prevention	Convergence	Attaining policy
K (native key)	any	yes, with a durable key, while every execution is within retention r	yes, within retention	key + recoverable identity
A, reversible	exact or lagged, Δ known	yes, by waiting $\Delta + \lambda$ before each re-issue (P3), except against redelivery	yes, without the wait, and also against redelivery that lands before the final count	recoverable tag + horizon + targeted compensation
A, reversible	lagged, heavy-tailed δ	no	per round, with probability at least $1-\sum_i p_i(1-F(H-\lambda-t_i))$ for independent attempts; no stopping policy beats the bound of P7	same; residual risk falls as H grows
A, irreversible	exact or lagged	only if Δ is known and waited out, and never against redelivery	equals prevention	tag + wait, then count; report redelivered duplicates
C, content only	content with look-alikes	no	no	safe abstention (a tag, where one exists, makes the write class A)
U, exclusive writer	aggregate	yes, by waiting $\Delta + \lambda$ before each re-issue, except against redelivery	yes, if fungibly reversible	delta + horizon
U, concurrent writers	aggregate	no	no	safe abstention

Table 2 summarises the propositions and yields three predictions that the experiments test. Idempotency keys and workflow engines should match Settle on class-K writes and fall short on class-A writes. Every method that recognises its own effects by content should fail under look-alikes. And no method should succeed in the last row; the best a method can do there is to say so.

5. How common is each contract? A census of 52 APIs

The theory makes the contract of a write, rather than the agent, decide what recovery can achieve. Whether this matters depends on how real services are distributed over the cells of Table 2. I therefore coded the write contracts of 52 widely used APIs from their official documentation. The census is exploratory; it was coded after the test runs and was not pre-registered (Section 7).

Sample and coding. I chose APIs that agents are commonly connected to, in four domains of twelve to fifteen APIs each: payments and commerce, developer and project tools, messaging and e-mail, and CRM, data and cloud services. For each API I examined the *create* operation an agent would typically call (for example, creating an issue, sending a message, inserting a record or creating a payment) and recorded six properties from the vendor's documentation. The first is whether the operation accepts a client-supplied key that the *server* deduplicates, and for how long the key is retained. The second is whether a client-controlled field is persisted with the created object, such as a metadata map, an external id, a custom field, a header or a label. The third is whether objects can be retrieved by the exact value of that field. Lookups are coded as exact, as a search that the documentation describes as fuzzy or eventually

consistent, as possible only by scanning a list of recent objects, or as impossible. The fourth is whether the field is hidden from end users. The fifth is whether the created effect can be undone through the API. The sixth is the URL of every documentation page relied on.

Each API receives a class from these properties: **K** if it has a server-side key; **A** if, without a key, it has a taggable field with an exact lookup; **A−** if the field can be found only through a lagged search or by scanning lists; and **C** if there is no usable field or no lookup at all. A second pass re-checked a random sample of 12 entries against the documentation, starting from the first pass's codes and sources. It confirmed all 12 class assignments and found 1 factual error in a detail that did not affect the class (an additional search endpoint for one payment API). Twelve agreements out of twelve bound the class error rate only loosely, at about 25% with 95% confidence, so individual entries should be read with their sources. The full coded table, with documentation links, is part of the released artifacts, and Appendix E lists it.

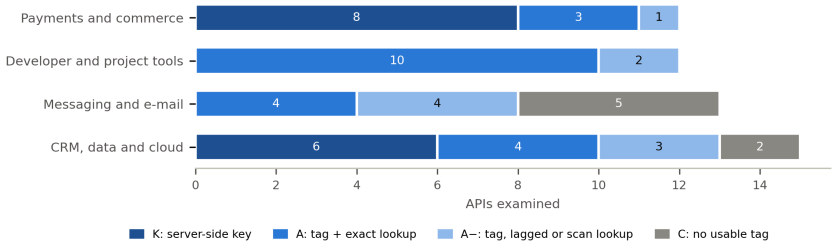


Fig. 2. Write contracts of 52 APIs by domain, coded from vendor documentation. K: the create operation accepts a server-deduplicated key. A: no key, but a client-controlled field with an exact lookup. A−: a field that can be found only through a lagged search or by scanning lists. C: no usable field or no lookup.

Keys are rare outside payments. Only 14 of the 52 APIs (27%) have a server-side key on the examined operation (Fig. 2). Eight of the twelve payment APIs have one; the other six are in the CRM, data and cloud group: Zendesk, Microsoft Graph calendar events, PagerDuty, and the conditional creates of S3, DynamoDB and Elasticsearch. None of the twelve developer and project tools and none of the thirteen messaging and e-mail services offers a key. Eleven key-less APIs have something *close to* a key, with weaker or undocumented guarantees: upserts on a client-chosen field (Salesforce, Airtable) without a documented concurrency guarantee; uniqueness constraints that reject a repeat with an error instead of returning the original outcome (Razorpay's order receipt, Confluence page titles, HubSpot's unique properties); client-chosen ids whose duplicate handling is undocumented (Linear) or whose collisions the provider explicitly does not guarantee to detect (Google Calendar); duplicate checks that last thirty seconds (Braintree) or minutes (Discord's nonce, SQS FIFO queues); and deduplication within one producer session (Kafka). An agent can build on several of these, but none is documented to give the guarantee of Proposition 1, and I do not count them as keys.

Keys expire. Retention is documented for most keys, and several are short. Mollie keeps keys for one hour, Zendesk for two and PayPal for six by default. PagerDuty's deduplication key lasts only while the alert is open, so a retry after it has been resolved opens a new incident. Square, Wise and Microsoft Graph do not document retention at all. By Proposition 1, a key protects only re-issues within its retention, and agents that resume after a crash, a deployment or a human approval can exceed one hour. Worse, five of the eight keyed payment APIs can find a payment by a client field only by scanning a list of recent payments, one (Adyen) offers no lookup at all, and one (Stripe) offers only an eventually consistent search. Once a key has expired, these APIs leave the client no exact way to attribute.

Most key-less APIs are taggable. Of the 38 key-less APIs, 34 accept a client-controlled field with the write, and 31 (82%) persist one that can also be looked up. Of these, 21 (55% of the key-less APIs) offer an exact lookup by the field, which makes them class A in the sense of Section 4; under a strict reading that treats Mailgun's events API and the Sent Items copy of Microsoft Graph mail as lagged, the count is 19. For class-A APIs Proposition 2 applies as soon as the effect is reversible: 14 of the 21 class-A operations can be undone through the API, and 2 more only partly (GitHub's REST API closes rather than deletes an issue, and Notion moves a page to the trash). An exact lookup on GitHub needs the tag as a label, one per operation; a tag in the issue body can be found only through the lagged search index. Most of the remaining class-A operations send e-mail, which cannot be recalled, so Proposition 3 applies to them. Ten APIs are class A−. Jira's and Freshdesk's searches, for example, are documented as possibly missing recent

writes (Freshdesk: for a few minutes), and Slack messages and Google Sheets rows can be found only by scanning recent items. Here attribution is possible, but only under a read lag λ that is sometimes undocumented, or at the cost of a scan.

A minority offers no attribution at all. Class C, with no usable tag field or no lookup, covers 7 APIs (18% of the key-less ones): Microsoft Teams channel messages, Twilio SMS, Telegram, WhatsApp, Intercom conversations, Google Pub/Sub and standard SQS queues. Their writes cannot be tagged, or a tag cannot be looked up. Under a strict reading, which also moves Discord and Amazon SES to class C because their lookups depend on undocumented or sampled data, the share is 24%. For class-C writes the theory leaves only safe abstention; waiting out a known in-flight bound tells the client that no attempt is still pending, but not whether one executed.

Tags need not be visible. A tag written into an issue body is visible to the people who read the issue. Among the 31 taggable key-less APIs, 9 offer a field the documentation describes as hidden from end users, for example Asana's external data, Shopify's metafields, Jira's issue properties, Postmark's and SendGrid's message metadata, and mail headers. Another 10 offer only visible fields, such as GitHub labels or a Notion property, and for 12 the documentation does not say.

What the census implies. The census supports the premise of this paper on documentary evidence. Prevention by a server-side key is available for about a quarter of the APIs I examined and concentrated in payments. Convergence by attribution is available for most of the rest. Only a minority of services, mostly messaging services, leaves nothing but abstention. The census has limits. It covers documented behaviour, not behaviour under fault, and the sample is purposive rather than random. It codes one create operation per API, and several "unclear" entries could move with better documentation. The experiments that follow test whether the documented contract classes behave as the theory predicts, on real services and under real faults.

6. Settle: recovery by attribution

Settle is a layer between an agent and its tools that turns the constructive side of Section 4 into a protocol. The agent, scripted or a language model, calls tools as usual. Settle intercepts every write and, in the language-model setting, every deletion the agent requests. It returns one of three observations: *ok* with the effect's id, *unknown* with a reason, or the service's own error when the outcome is known. Settle has six parts.

Operation identity. Each logical intent receives one operation id, which is kept across retries, restarts and rephrasings. For the scripted agent, the intent is the plan step. For a language-model agent, a write is bound to an intent by its *intent fields*: the title of an issue, the recipient and subject of a mail, the item of a record. A rephrased re-issue of the same issue therefore maps to the same operation. In my implementation this binding reuses the harness's mapping of model writes onto the task's plan steps by these fields (Appendix C). A deployed layer has no plan to map onto and would bind by the intent fields alone. It would then also merge two deliberately identical writes, such as the same reminder sent twice on purpose, unless the agent marks them as distinct. The follow-up study runs this plan-free binding (S-np) and finds identical outcomes (Section 9).

Durable intent journal. Before the first dispatch, Settle appends `intent → {op id, arguments, aggregate before}` to a journal and calls `fsync`. Each attempt moves the record through the states `dispatching`, then `confirmed`, `in_doubt` or `unknown`. After a crash, a record found `dispatching` or `in_doubt` is resolved before anything is re-issued, which is what Proposition 6 requires.

Attribution. Settle uses the strongest attribution that the contract offers. For a class-K write, the op id is the idempotency key: TigerBeetle derives the transfer id from it, and the object store uses a conditional create. For a class-A write, the op id is written into a field that the service persists, such as an `op-id:` line in an issue or comment body, the mail `Message-ID` header, a reference column, a stream field or object metadata. The agent is not told about the tag, although it can see it when it reads the record back. For a class-U write, Settle records the aggregate before the write and uses the change, which is valid only under exclusivity (Proposition 5). A write that can be found only by content (class C in Table 2; in my stack, the bulk e-mail) is not attributable. For such writes the implementation falls back to a content match, as the baselines do, and lists the operation as uncertain whenever it had to re-issue it. It does *not* abstain when the content match succeeds at the first check, which is the case a look-alike exploits; Section 8.5 reports the consequence. A design that abstains on every ambiguous class-C write would report these failures, at the cost of reporting uncertainty in every ambiguous class-C episode; the follow-up study measures it (S2).

Resolution of unknown outcomes. On an error or timeout, Settle re-issues a keyed write with the same key. For a

taggable write, it waits out the read path's documented lag and counts the effects carrying the tag. If there are one or more, the operation is done and the id of the kept effect is returned. If there are none, the write is re-issued under the same tag. For an irreversible write such as an e-mail, Settle first waits until the assumed in-flight bound Δ has passed since the last attempt, so that a re-issue cannot overtake a late commit (Proposition 3). For an aggregate shared with other writers it stops and returns *unknown*. After three attempts it returns *unknown* in every case.

Reconciliation horizon. When the agent finishes, Settle keeps reconciling every operation that was ever in doubt until $t_{\text{last}} + \Delta + \lambda$. For each such operation it counts the effects by tag, re-issues if none exists, and marks surplus effects for compensation. The final sweep also covers operations that were confirmed at once, which catches transport-level redelivery. If no operation was ever in doubt, the sweep runs at once without waiting, so it catches a redelivery only if the duplicate is already visible when the agent finishes. My main redelivery fault delivers the duplicate immediately; a delayed redelivery, as from a queue's visibility timeout, needs the sweep to wait in every episode, which the follow-up study tests (S2).

Evidence-gated compensation. A surplus effect is neutralised only if it carries the operation's tag and is not the effect that the operation keeps, which is the one whose id later steps have already received. Compensations are themselves writes. Settle's compensations in my stack are deletions, which are idempotent, and decrements of exclusively written aggregates, which are not. In the experiments every compensation travelled on a fault-free path, so assumption (e) of Proposition 2 was not stressed. Three further consequences follow. An effect that cannot be attributed is never compensated, which avoids the *orphaned compensation* anomaly [9]. An irreversible surplus is reported as an unrecoverable duplicate instead of being hidden. And in the language-model setting, the same rule guards the agent's own deletion tools: a deletion of an effect that carries none of the task's tags, or of the only effect of a completed operation, is refused with a short explanation. I added this rule on the development split, after observing that a careful agent which sees two identical issues "cleans up" by deleting the other user's issue (Section 8). Clean-up by content is exactly the unsound compensation of Proposition 4.

The pseudocode below summarises the write path for a taggable, reversible tool; the released code contains the full implementation.

```
write(intent, args):
  rec = journal.get(intent)
  if rec is None:
    rec = journal.put(intent, op=new_id(), state=intent)
  elif rec.confirmed:                     # done before a crash or re-issue
    return ok(rec.value)
  elif rec.state in (dispatching, in_doubt):    # restart after a crash:
    return resolve(intent)       # wait lag (+ Δ if irreversible), count
  journal.put(intent, state=dispatching)  # write-ahead, before sending
  r = dispatch(args + tag(rec.op))
  if r.ok: return ok(journal.confirm(intent, r.id))
  journal.put(intent, state=in_doubt)
  sleep(lag(tool))
  found = read_by_tag(rec.op)             # attribution, not content
  if found: return ok(journal.confirm(intent, kept(found)))
  return write_again(intent)        # same tag; ≤ 3 attempts, then unknown

finish():
  if journal.ever_in_doubt():
    sleep_until(max(last_attempt) + Δ + λ)    # reconciliation horizon
  for op in journal.ever_in_doubt() ∪ journal.confirmed():
    found = read_by_tag(op)
    if not found: re-issue(op)
    for e in found − {kept(op)}:
      compensate(e)                   # only effects carrying op's tag
  report(unknown = unattributable re-issued ops, shared aggregates,
         irreversible duplicates)
```

Assumptions and parameters. Settle does not assume an idempotency key on the service, a well-behaved agent (the agent may retry, rephrase, crash or try to delete things), or a known order of faults. It does assume that the tag survives byte-exact in the persisted field, assumption (b) of Proposition 2, which the environment validation checks per service,

and that no one else copies the tag into another record, assumption (g). A tag in a visible body line can be copied, for instance when a user quotes the record; requiring the tag *and* the client's own author account before counting or compensating an effect would close most of this gap, and my implementation does not yet do so. Its parameters are the assumed in-flight bound Δ and the documented read lags. Δ was set on the development split to 20 s and frozen before testing. Where it is violated, Propositions 2 and 7 predict the residual risk; the experiments include a heavy-tailed delay distribution to measure that risk, and Section 8.6 traces how it moves with Δ.

Ablations. Four ablations each remove one component. S−j keeps the intent journal, and with it the tag, in memory, so by Proposition 6 it tests whether the tag must survive a crash; S−t matches effects by content instead of by tag; S−h verifies once and has no horizon; and S−c flags surplus effects without compensating them. Each ablation should fail in the cells where the theory says its component is needed, and should be close to Settle elsewhere.

7. Experimental design

The design is a paired factorial. Every *world* is replayed under every recovery condition of a driver. A world fixes the task instance, the focal write that receives the fault, the fault mode and the read model the agent sees, so all comparisons between conditions are within-world. The study was pre-registered internally: hypotheses, analysis code, task definitions, prompts and the SHA-256 hashes of all source files were frozen before the first test episode, and the test presets refuse to run if any hashed file changes (Appendix D). The freeze is recorded in the released repository rather than with a third-party registry.

7.1 Services, tools and contracts

All services run unmodified in containers on one workstation: PostgreSQL 16 for records and stock levels; Gitea 1.24 [36] for issues and comments, with its real asynchronous search indexer; Mailpit [37] for SMTP with a search API; Redis 7 streams and counters [38]; SeaweedFS [39] as an S3 object store; and TigerBeetle [18] for ledger transfers. The agent reaches them through thin tool adapters (Table 3), each a few lines over the service's own API. Where Settle writes a tag, it uses a field of the kind that the census found in real APIs: a body line as in GitHub issues, a mail header as in Gmail, a reference column as in Salesforce's external ids, a stream field as in Kafka headers. The stack covers every row of the achievability map.

Table 3. Tool contracts. Class K has a native key, class A is key-less but taggable, class U exposes only an aggregate, and class C can be found only by content.

Tool	Service	Class	Reversible	Tag field or key	Read paths
create issue	Gitea	A	targeted (delete)	`op-id:` line in body	list + body (exact); search (lagged)
add comment	Gitea	A	targeted	tag line in body	comment list
send e-mail	Mailpit SMTP	A	none	client `Message-ID`	search by header or content
send bulk e-mail	Mailpit SMTP	C (content only)	none	none	search by content
insert record(s)	PostgreSQL	A	targeted	`reference` column	select
append event	Redis stream	A	targeted (XDEL)	`op_id` field	range scan
adjust stock	PostgreSQL	U	fungible (inverse)	none	sum only
increment counter	Redis	U	fungible (decrement)	none	value only
create transfer	TigerBeetle	K	targeted (reversal)	128-bit transfer id	lookup by id
put object	SeaweedFS S3	K	targeted	key + `If-None-Match: *`	head

Tasks. Eight task templates (Table 4) define short workflows of two to four writes across services, one of which (T5) inserts a batch of six records. Each comes with an exact specification of the final state: every write must leave exactly one standing effect, and every aggregate must change by exactly the required amount. The development split consists of five templates on instance 0, and every method parameter was set on it. The test split uses all eight templates on fresh instances 100–102. Three templates (T3, T6, T8) were not used during development; for these, only a fault-free and error check of the harness was run, on an instance outside the test set.

Table 4. Task templates. Test-only templates: T3, T6, T8.

Template	Workflow	Writes (class)	Stresses
T1	open an incident	issue (A), event (A), page e-mail (A, irreversible), record (A)	mixed reversibility
T2	pay out a batch	2 transfers (K), record (A), e-mail (A, irr.)	keyed and key-less in one task
T3	ship a release	object (K), issue (A), comment (A), bulk e-mail (C, irr.)	content-only irreversible write
T4	receive stock	2 stock adjustments (U), counter (U), record (A)	aggregate reads
T5	migrate a batch	6 records in one non-atomic batch (A), event (A)	partial failure
T6	send customer notices	3 e-mails (A, irr.), issue (A)	irreversible-heavy
T7	triage a ticket	issue (A), 2 comments (A), record (A)	dependent steps, look-alike prone
T8	refund a customer	transfer (K), record (A), e-mail (A, irr.)	keyed payment + notification

7.2 Faults and read models

Network and process faults. The faults are real. They are injected on the client path with Toxiproxy [40], through one proxy per worker, armed by a hook immediately before the focal write's bytes are sent. In *lost_ack*, the request is delivered and committed and the reply is cut. In *not_exec_err*, the connection is cut before the request is delivered, and the agent sees the same error as in lost_ack. In *held*, the reply is withheld past the tool deadline after the request has committed; in *not_exec_to*, the request is held upstream and dropped, and the agent sees the same timeout. In *late*, the request reaches the service 12 s after dispatch, long after the 4 s tool deadline has expired. *late_tail* is like late but draws the delay per world from a log-uniform distribution on 5–120 s, so that in part of the worlds it exceeds Settle's assumed bound of 20 s. In *partial*, a non-atomic batch is cut after its third row. In *redelivery*, the transport delivers the same request twice. In *crash_after* and *crash_before*, the agent process exits (`os._exit`) right after or right before the focal dispatch and is restarted from its checkpoint.

Each pair of modes (lost_ack and not_exec_err, held and not_exec_to, crash_after and crash_before) gives the agent the same observation with opposite hidden outcomes. Transport errors are normalised to one exception type and message, so the pairs are observation-equivalent by construction. The two network pairs make the design symmetric in what it penalises, though not in size (held occurs in 42 of the H1 worlds and not_exec_to in 96; Table 6): a method that re-issues too eagerly fails in one mode of each pair, and a method that re-issues too reluctantly fails in the other. The crash pair is not symmetric in this sense. After a crash before dispatch, the driver's checkpoint re-issues the step in every condition, so crash_before ends consistent everywhere, and the pair penalises only eager re-issuing. An environment validation checks on every service, from the service's own records, that each mode produces its intended hidden outcome; 493 of 510 trials did, and the exceptions are TigerBeetle's not_exec modes, whose client library retries internally (Appendix B).

Read models. The read model is varied per world. Under *exact* reads the agent has the default read paths. Under *lagged* reads it can find issues only through Gitea's asynchronous search index. Under *look-alike*, a background actor creates an effect with content identical to the focal write at the moment of the fault, just before the focal request is sent, through the same service account. This is the worst case for any check by content, since no author or creation-time field separates the two records; a look-alike from another account, or from an earlier time, could be told apart by those fields. Under *shared aggregate*, a background actor adjusts the same counter or stock level.

7.3 Conditions

Every condition sits between the driver and the tools, and only the recovery logic differs.

- **C0 plain:** the agent's own behaviour; for the scripted driver, one immediate re-issue on error.
- **C1 never retry:** stop and report at the first failed write.
- **C2 SDK retry:** transparent client retries, three with back-off.
- **C3 idempotency keys:** a key per call, generated in memory and passed wherever the service honours one, which is common client practice [5].
- **C4 workflow engine:** each write is a Temporal activity with a retry policy of three attempts, and keys are derived from the durable workflow and activity ids [12].
- **C5 verify-before-retry:** re-issue only if a content-based postcondition fails [23].
- **C6 guard:** my re-implementation of Li's client-side guard from its published description [8], with automatic keys, a re-check after the documented lag, a block on repeats it cannot verify, and an uncertainty annotation in

the observation.

- **Gw guard + wait:** C6, but waiting the same in-flight bound Δ as Settle. This matched-latency control shows whether waiting alone explains Settle's results.
- **C7 reflection:** after a failed tool call the model is asked to reason about the failure before acting [41]; language-model drivers only.
- **C8 rollback:** on failure, compensate the completed steps from a log, then restart the workflow [14], [15].
- **S Settle**, and its ablations S−j, S−t, S−h and S−c.
- **O outcome oracle:** knows the hidden outcome that the fault mode is designed to produce for the focal write, including when an in-flight request should land, and re-issues only if the write is designed never to execute. It does not compensate. It shows what perfect knowledge of the focal outcome achieves *without* compensation; it is not a lower bound for methods that compensate, and Settle does better than it where compensation matters.

Retry budgets are small and comparable. The scripted driver re-issues a failed step once, C2 retries up to three times, and the guards and Settle make at most three attempts per write. Language-model conditions share the same model, prompts, and limits of 30 turns and 40 tool calls. Wall-clock time is reported, not capped.

Limiting bias towards my own method. Because I designed Settle, the theory and the harness, I took several precautions against a comparison that favours Settle by construction. The predictions of the achievability map that the hypotheses test, including where Settle itself must fail, were frozen with the analysis code before the test runs. Each baseline family is represented by its strongest configuration that I could build without attribution: durable rather than in-memory keys (C4), a guard that waits as long as Settle (Gw), and an outcome oracle. Variants that *do* use the tag but lack another component are the ablations, and I report them next to the baselines so that the contribution of attribution can be separated from that of the journal, the horizon and compensation. Three of the eight test templates were not used during development. The network fault pairs penalise eager and reluctant re-issuing equally. And grading is done by code from the services' own records, independently of every recovery layer. These precautions do not remove the main limitation: the baselines are denied tags by design, so part of the comparison confirms Proposition 4 rather than testing an engineering choice. The re-implemented baselines may also differ from their authors' code in detail; I release mine and describe the behaviour of each.

7.4 Drivers

Scripted driver. The scripted driver is a deterministic agent without a model. It executes the plan in order, keeps a durable step checkpoint as agent frameworks do, re-issues a failed step once, finds earlier results by content when it needs them, and reports honestly what it believes. It isolates the effect of the recovery layer and makes the large grid possible.

Language-model drivers. Three OpenAI models run, each at its default reasoning setting, through the Responses API [42]: `gpt-5.6-luna`, which reasons by default; `gpt-5.4-nano`, which does not; and `gpt-4.1-mini`, which is not a reasoning model. The language-model study does not aim to rank models. It asks two questions that the scripted driver cannot answer: what a model does on its own after an ambiguous write, and whether Settle's guarantee survives an agent that rephrases writes, deletes records, waits, and crashes in the middle of a conversation. Each model receives a system instruction that describes it as an autonomous operations agent; the task goal, with the exact required strings and the sentence "Each action must take effect exactly once."; and the tools of its task, namely writes, reads, its own deletion tools, a wait tool and finish (Appendix C). The agent ends by calling finish with a status, a summary and a list of operations whose outcome it is unsure of. The conversation is checkpointed after every tool result. After a crash, the run resumes from the checkpoint and re-executes any pending tool call, which is the replay behaviour of common agent frameworks.

The language-model grid uses the first two writes of every template on test instance 100; the modes none, lost_ack, not_exec_err, late, late_tail and crash_after; all four read models where they apply; and conditions C0, C5, C6, C7 and S. `gpt-5.6-luna` runs two repeats and the other models one, for 2,680 planned episodes. Every model response is cached by a hash of the request, so the released cache replays every decision without an API key.

7.5 Grading, statistics and pre-registration

Grading. Grading is done by code from the services' own records, read with separate credentials after quiescence: PostgreSQL audit triggers; the Redis stream's monotonic `entries-added` counter and an atomic counter-audit list; TigerBeetle's immutable transfer history; Mailpit's capture store, which no tool can delete from; and Gitea state together

with a compensation log written by the harness. Background-actor effects are identified from the actor's own log. Neither agents nor recovery layers can import the grader.

Statistics. Rates carry cluster-bootstrap 95% intervals that resample template × instance clusters 10,000 times. Paired comparisons use the exact McNemar test with Holm correction within each hypothesis, and equivalence uses two one-sided tests through a 90% interval. Because the scripted driver is deterministic and the instances of a template differ mainly in their strings, worlds are not independent draws, and the intervals measure replication across my design grid rather than uncertainty about a population of deployments. As a sensitivity check I also resample the eight templates as clusters (Section 8.2). The rates themselves are averages over a designed grid of worlds, so I report them per fault mode and per contract cell as well as pooled. The six hypotheses, their thresholds and every analysis deviation are listed in Appendix D, and Section 8 reports every hypothesis, confirmed or not. Two analyses are exploratory and were not pre-registered: the API census (Section 5) and the horizon trade-off (Section 8.6). I label them as such wherever they appear.

8. Results

8.1 The test runs

The test split comprises 17,820 scripted and 2,680 language-model episodes. Every scheduled episode finished, so no record is excluded. The language-model test runs cost US$ 3.24, and the main study as a whole, including its pilot and development checks, cost US$ 3.40. With the follow-up study (US$ 6.09, Section 9), all model calls of the project cost US$ 9.49. Two amendments were made after the freeze (Appendix D). The first fixed a lock race in the cost ledger during the runs and could not change any outcome. The second fixed a defect found in the audit before writing: transfer keys chosen by the models collided across episodes that replay the same task, so the ledger refused the transfer while the tool reported success. All affected language-model episodes were re-run under the amended code. The verdicts of H1–H4 and H6 did not change, H5's cost criterion for `gpt-5.4-nano` moved from inconclusive to confirmed, and Section 8.9 also reports the numbers from before the amendment. Every condition left a consistent final state in every one of the 1,305 fault-free scripted episodes, so the harness and the grader create no inconsistency of their own.

8.2 Settle converges where convergence is possible (H1)

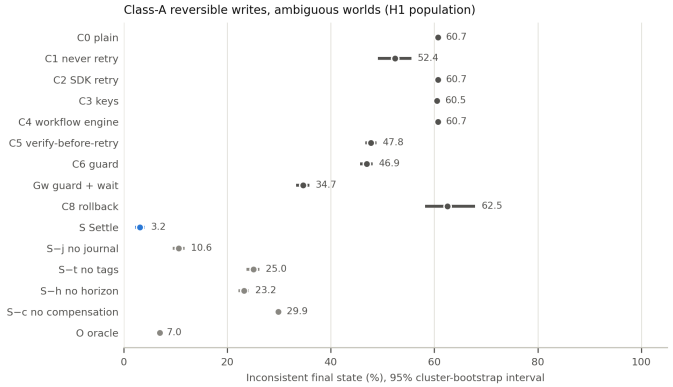


Fig. 3. Inconsistent final states for class-A, targeted-reversible writes in ambiguous worlds (the H1 population; scripted driver, all read models). Points are rates; bars are 95% cluster-bootstrap intervals over template × instance clusters.

Fig. 3 shows the H1 population: ambiguous worlds whose focal write is taggable and reversible. Settle leaves 3.2% of these episodes inconsistent (95% CI 2.2%–4.1%). The best baseline is Gw guard + wait, the guard that waits as long as Settle, at 34.7%, and the paired difference is −31.5 pp (pp: percentage points; 95% CI −32.5 pp to −30.5 pp; Holm-adjusted exact McNemar $p < 0.001$). H1 is confirmed.

Plain retries, client retries, in-memory keys and the workflow engine all end at about the same level: 60.7%, 60.7%, 60.5% and 60.7%. None of them can tell whether an unacknowledged write took effect, and a key or a durable workflow history does not help when the service ignores the key. Verifying by content helps, but only partly: verify-before-retry reaches 47.8% and the guard 46.9%. Waiting out the in-flight bound lowers the guard to 34.7%, so waiting alone does not explain the difference between the guard and Settle. Rollback is the worst condition at 62.5%: it compensates and

restarts on every error, including errors after which the write *had* taken effect, so the restarted workflow duplicates it. The oracle knows every hidden outcome but cannot remove a duplicate, and ends at 7.0%. All its failures in this population are transport redeliveries, which no knowledge of outcomes prevents and only compensation repairs. This is why Settle does better than the oracle in this population; without redelivery and partial batches the oracle is perfect (Table 6).

Table 5. All ambiguous worlds by condition, scripted driver. "Duplicate executed" counts episodes in which any step's effect was executed more than once, *including* duplicates that were later compensated, and "missing" episodes with at least one missing step. This is the prevention metric of prior work [8]. It shows why prevention and convergence differ: Settle allows transient duplicates and removes them.

Condition	n	Inconsistent %	Duplicate executed %	Missing %	Collateral %	Overclaim %	Mean time (s)
C0 plain	1101	60.8	60.8	0.0	0.0	60.8	3.3
C1 never retry	1101	46.5	14.2	32.3	0.0	14.2	3.1
C2 SDK retry	1101	55.9	55.9	0.0	0.0	55.9	3.7
C3 keys	1101	55.8	55.8	0.0	0.0	55.7	3.3
C4 workflow engine	1101	55.0	55.0	0.0	0.0	55.0	7.4
C5 verify-before-retry	1101	46.6	32.5	14.2	0.0	46.6	3.6
C6 guard	1101	42.8	27.0	15.8	0.0	39.5	3.5
Gw guard + wait	1101	33.7	17.9	15.8	0.0	30.4	14.1
C8 rollback	1101	63.9	83.7	0.0	3.3	63.9	4.0
S Settle	1101	7.4	22.0	2.2	0.0	3.4	18.5
S−j no journal	1101	15.0	29.6	2.2	0.0	10.7	16.3
S−t no tags	1101	22.7	18.3	17.4	0.0	18.6	14.8
S−h no horizon	1101	23.9	27.1	2.6	0.0	19.7	4.8
S−c no compensation	1101	24.2	22.0	2.2	0.0	3.4	18.5
O oracle	1101	6.8	6.5	0.3	0.0	6.8	6.7

Table 5 extends the comparison to all ambiguous worlds. Settle's rate of duplicate executions (22.0%) is higher than its inconsistency rate (7.4%); the difference is the duplicates that Settle let happen and then removed. Under the prevention metric of prior work Settle looks mediocre. Under convergence it is the only condition close to the oracle.

Where the difference comes from. A pooled rate over a designed grid depends on the mix of fault modes, so Table 6 breaks the H1 population down by mode and adds the ablations. Three of them (S−h, S−c, S−j) use tags but lack another component; S−t keeps every component but matches by content. Two modes favour Settle by construction: every condition without a final sweep fails on redelivery, and every condition that does not check rows individually fails on a partial batch. Without these two modes, Settle leaves 3.4% of 558 worlds inconsistent against 29.4% for the waiting guard, and the oracle is perfect. The ablations separate the components. A tag with a journal and compensation but without the horizon (S−h) reaches 25.1%, about the level of the waiting guard. A tag with a horizon but without compensation (S−c) reaches 24.7%, and a tag that is lost in a crash (S−j) reaches 11.5%. Attribution is therefore necessary but not sufficient: a tag-aware variant that lacks the horizon or compensation is not much better than the waiting guard, as Proposition 2 leads one to expect. These ablations remove a component from Settle; they are not the strongest tag-based alternative. Proposition 3 implies that a tag-based *wait-then-count* policy, which waits out Δ before every re-issue and never compensates, prevents duplicates whenever Δ holds. The follow-up study (Section 9) runs exactly this policy and finds it identical to Settle on every ordinary fault, so Settle's advantage over it lies mainly in transport redelivery. The comparison with the waiting guard does not depend on the clustering. Resampling the eight templates instead of the 24 template × instance clusters gives a paired difference of −31.5 pp (95% CI −33.2 pp to −29.6 pp), or −26.0 pp (−27.3 pp to −24.3 pp) without redelivery and partial batches, and Settle is better than the waiting guard within every one of the eight templates (a sign test over templates gives $p = 0.008$, two-sided).

Table 6. The H1 population by fault mode: inconsistent final states (%), scripted driver, all read models. S−h, S−c and S−j use tags but lack the horizon, compensation and a crash-surviving journal respectively; S−t has every component but matches by content instead of by tag. The last row excludes the two modes that every condition without a final sweep or row-level checks fails by construction.

Fault mode	worlds	C0	C4	C6	Gw	S−h	S−c	S−j	S−t	S	O
lost_ack	96	100.0	100.0	7.3	6.3	0.0	0.0	0.0	43.8	0.0	0.0
not_exec_err	96	0.0	0.0	43.8	43.8	1.0	0.0	0.0	43.8	0.0	0.0
held	42	100.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
not_exec_to	96	0.0	0.0	43.8	43.8	4.2	0.0	0.0	43.8	0.0	0.0
late	96	100.0	100.0	62.5	6.3	99.0	100.0	0.0	6.3	0.0	0.0
late_tail	42	100.0	100.0	100.0	54.8	95.2	100.0	45.2	45.2	45.2	0.0
crash_after	45	100.0	100.0	100.0	100.0	0.0	0.0	100.0	0.0	0.0	0.0
crash_before	45	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
redelivery	42	100.0	100.0	100.0	100.0	0.0	100.0	0.0	0.0	0.0	100.0
partial	3	100.0	100.0	100.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0
all but redelivery and partial	558	57.5	57.5	42.7	29.4	25.1	24.7	11.5	27.1	3.4	0.0

8.3 Where keys and workflow engines are enough, and where they are not (H3)

Table 7 fills in the achievability map with measured rates. On class-K writes, where the service honours a key, Settle and the workflow engine with durable keys are both exact: neither leaves any of the 108 paired worlds inconsistent, so the equivalence margin is met trivially. H3 is confirmed. The oracle's three failures in this cell (2.8% of the 108 keyed worlds) are late-tail worlds in which the delayed TigerBeetle transfer, injected to commit after 50–84 s, never executed; conditions that re-issued with the same id ended consistent there. This is a fault that did not realise its intended outcome, and it affects only the oracle, which by design never re-issues a write it expects to land. Keys generated in memory (C3) fail in exactly the crash worlds (37.5% inconsistent) and nowhere else (0.0%), as Proposition 1 predicts.

On key-less writes the picture reverses. Keys and the workflow engine are no better than plain retries, because there is nothing to send the key to. Over all key-less ambiguous worlds their rates are 60.9% and 61.0%, against 61.0% for plain retries. The workflow engine's durable history does not stop a crashed step from taking effect twice: in key-less crash-after-dispatch worlds it is inconsistent in 100.0% of episodes, against 0.0% for Settle, because resuming the step means executing the activity again (Proposition 6). In every key-less cell Settle has the lowest rate of the non-oracle conditions or ties for it, and its remaining failures in the attributable, reversible cells are all delays beyond Δ.

Table 7. The achievability map, measured: inconsistent final states (%) by contract cell of the focal write and condition, in ambiguous worlds (scripted driver). "Worlds" is the number of ambiguous worlds per condition.

Contract cell (focal write)	worlds	C0	C3	C4	C5	C6	Gw	C8	S	O
K, keyed	108	58.3	8.3	0.0	41.7	8.3	8.3	58.3	0.0	2.8
A, reversible, exact read	387	66.7	66.4	66.7	45.0	45.2	29.2	60.2	4.9	10.9
A, reversible, lagged read	48	50.0	50.0	50.0	37.5	25.0	0.0	62.5	0.0	0.0
A, reversible, look-alikes	168	50.0	50.0	50.0	57.1	57.1	57.1	67.9	0.0	0.0
A, irreversible (mail)	234	61.5	61.5	61.5	46.2	45.7	36.3	71.8	12.8	7.7
C, content only, irreversible	39	61.5	61.5	61.5	46.2	46.2	35.9	61.5	28.2	7.7
U, exclusive aggregate	81	66.7	66.7	66.7	44.4	44.4	44.4	66.7	3.7	11.1
U, shared aggregate	36	50.0	50.0	50.0	50.0	50.0	50.0	50.0	50.0	0.0

8.4 Content-based verification under look-alikes (H2)

When another user of the same account creates a look-alike, every method that recognises its own effects by content fails in more than half of the 252 ambiguous look-alike worlds of all contract classes (Table 7 restricts them to class-A reversible writes, 168 worlds): verify-before-retry in 54.8%, the guard in 54.8%, and Settle without tags in 69.0%. When the agent's write never executed, these methods take the other user's record for their own and skip the write, so their own effect is missing. Settle's rate is 2.4%, and H2 is confirmed. Rollback is the only baseline that deletes, and it causes collateral damage in 14.3% of these worlds. Every case is a world whose focal write was a comment: after the comment failed, rollback compensated the completed steps by deleting the agent's own issue, and with it the other

user's comment on that issue. Compensation that is correct for the agent's own effect still destroys what others attached to it.

8.5 What no method can do, and what Settle does instead (H4)

When another writer adjusts the same counter or stock level, every non-oracle condition is inconsistent in at least 50.0% of ambiguous worlds (H4a confirmed). Settle abstains there: it returns *unknown* instead of guessing. Across shared-aggregate and irreversible worlds, Settle overclaims in 4.9% of its episodes (15 of 309; H4b confirmed), and it reports a problem in 44 of the 59 episodes that ended inconsistent. Plain retries, by comparison, overclaim in 60.8% of all ambiguous worlds. In irreversible worlds every condition, the oracle included, shows duplicates or missing effects (H4c confirmed). H4a and H4c, like the lenient thresholds of H5 below, confirm properties that the design of the fault grid makes very likely: half of each shared-aggregate pair is a never-executed world, and redelivery of an irreversible write cannot be undone. They check that the harness behaves as the theory says rather than testing Settle.

Every one of Settle's failures is accounted for by the theory. Of Settle's 1,101 ambiguous scripted episodes, 81 ended inconsistent (Table 8). Each is either a violation of the in-flight bound that Settle assumes, whose consequence Propositions 2 and 7 bound, or falls into a cell where the theory admits no client-side guarantee. The first kind is not a prediction in the strong sense; it is a failure of an assumption, and the theory says how likely it is.

Table 8. Every inconsistent Settle episode (scripted driver), by the theoretical reason it could not converge, and how many Settle itself reported as uncertain or as an unrecoverable duplicate.

Cause (proposition)	Settle failures	of which reported
assumed in-flight bound violated: delay longer than Δ (risk bounded by Propositions 2 and 7)	36	5
transport redelivery of an irreversible key-less write (no client policy can prevent it; Proposition 3)	21	21
content-only write under a look-alike, not attributable (Proposition 4)	6	0
shared aggregate, not attributable (Proposition 5)	18	18
anything else	0	–

The late-tail worlds test the qualitative prediction of Proposition 2 that Settle can fail only when the in-flight bound is violated. Of 84 late-tail worlds, 48 had a delay above the assumed bound of 20 s. Settle failed in 36 of those and in 0 of the worlds whose delay was within the bound; the shortest delay among the failures was 21.2 s. Of the twelve worlds above the bound in which Settle did not fail, seven had a keyed focal write, which the bound does not affect, and in the other five the late commit still landed before Settle's final sweep at the end of the episode. Within the late-tail worlds Settle therefore fails only when its assumed bound is wrong (36 of the 48 worlds above the bound, none of the others). Of the 36 delay failures, 22 concern reversible and 14 irreversible writes. Settle's unreported failures are all of two kinds: 31 delay failures and the 6 content-only failures. The content-only failures are a gap in my implementation rather than in the theory: Settle resolves an ambiguous content-only write by content (Section 6), and a look-alike satisfies the check. A Settle that never confirms a content-only write by content reports them all, as the follow-up study confirms (Section 9). When a late commit lands after the horizon, Settle has already confirmed the operation, so only 5 of the 36 delay failures were flagged. In contrast, every shared-aggregate abstention (18 of 18) and every unrecoverable duplicate (21 of 21) was reported.

8.6 How the remaining risk trades against latency

Settle has one parameter that the theory does not fix, the assumed in-flight bound Δ. A larger bound buys a lower residual risk at the cost of latency, and Proposition 7 says that no smarter rule escapes this trade-off at a fixed decision time. This analysis is exploratory. To trace the trade-off without new runs, I use a property of Settle's decision rule that the recorded episodes let me check. In a late-commit world, Settle ends inconsistent if and only if the commit lands after its last decision about the operation. For an irreversible write that decision is taken Δ after the last attempt, which in these worlds is the first; for a reversible one it is the final sweep at the end of the episode. At the frozen Δ = 20 s, this rule reproduces the measured outcome in 167 of the 168 recorded late and late-tail worlds with exact reads, which is the population I replay. (It does not apply to shared-aggregate worlds, where Settle abstains.) Each recorded world therefore also tells me its outcome under a different bound: the decision time shifts by the difference in Δ, while the recorded delay of the commit stays the same. The replay assumes that Settle's other actions do not change with Δ, which holds for its decision rule but was verified only at the frozen value.

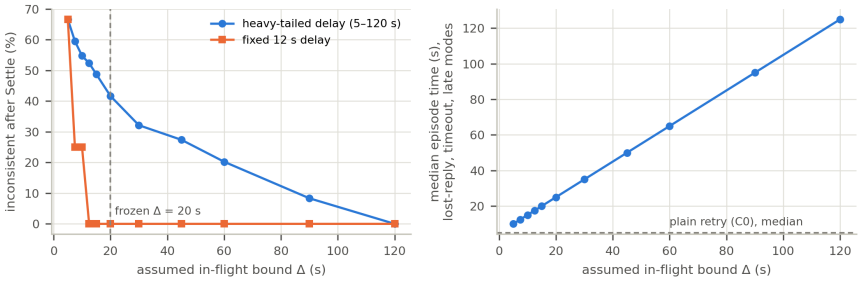


Fig. 4. Exploratory replay of the recorded Settle episodes under other assumed in-flight bounds Δ (scripted driver). Left: inconsistent final states in the late-commit worlds with exact reads, for the fixed 12 s delay and the heavy-tailed 5–120 s delay. Right: median duration of Settle episodes in the lost-reply, timeout and late-commit modes. At the frozen $\Delta = 20$ s the replay reproduces the measured outcome in all but one of these 168 worlds.

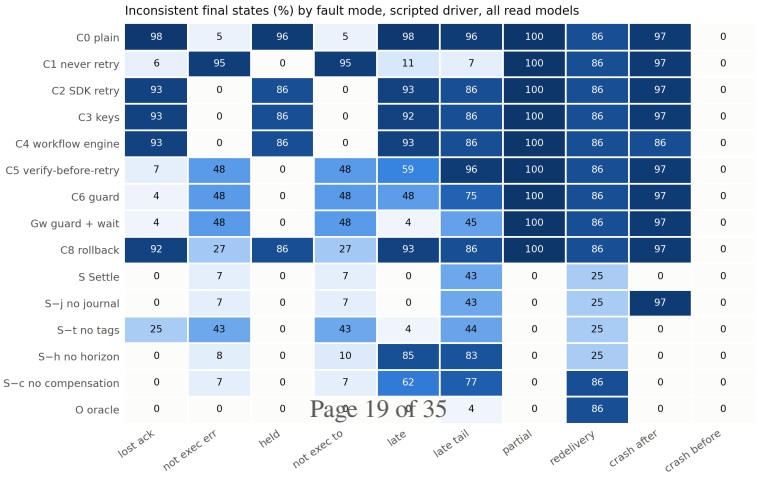
Fig. 4 shows the result. With the fixed 12 s delay, a bound shorter than the delay is a cliff: at $\Delta = 10$ s, 25% of these worlds end inconsistent, and from $\Delta = 12.5$ s on none does. With the heavy-tailed delay the trade-off is gradual. The replayed inconsistency falls from 55% at 10 s through 42% at 20 s (measured: 36 of 84, one world more than the replay) and 20% at 60 s to 0% at 120 s, the upper end of the distribution I chose. Over the same range the median duration of an episode in the lost-reply, timeout and late-commit modes grows from 15.0 s to 125.0 s, against 5.0 s for plain retries; only the episodes that actually waited out the bound move with it. The curve is the tail of the delay distribution seen through Settle's decision times, exactly as Propositions 2 and 7 predict. Two consequences follow. A deployment should set Δ from the delay tail of its own services. Settle's journal records every late commit that a sweep catches, but those observations are censored at the horizon, so the part of the tail that matters most is visible only if reconciliation continues beyond it. And since the horizon of a reversible write can continue after the agent has reported, the latency cost needs to be paid in full only for irreversible writes (Section 10).

8.7 Each component is needed where the theory says (ablations)

Table 9. Ablations. Δ is the ablation's inconsistency minus Settle's, paired by world, in the cell where the removed component is needed and in all other ambiguous worlds.

Ablation	Component removed	Pairs in cell	Δ in cell (pp)	95% CI	Δ outside (pp)	Verdict
S−j no journal	journal (P6)	87	+96.6	[+92.6, +100.0]	+0.0	confirmed
S−h no horizon	horizon (P2)	252	+70.2	[+64.7, +75.5]	+0.6	confirmed
S−t no tags	tags (P4)	462	+36.4	[+32.3, +40.3]	+0.2	confirmed
S−c no compensation	compensation (P2)	216	+74.5	[+65.4, +82.4]	+2.7	confirmed

Each ablation fails in its predicted cell and almost nowhere else (Table 9, Fig. 5). Without the journal, Settle fails after a crash. Without the horizon, it fails on late commits. Without tags, it fails under look-alikes, where it mistakes the other user's record for its own. And without compensation, it leaves standing the duplicates it detected. Outside its cell, no ablation is more than three points worse than Settle, although the small excess of S−c (+2.7 points) is itself significant.



`5.6-luna`, the only model that reasons by default, re-issues at once in only 17%, and its first action is a read in 69%. The hidden truth changes what the models do only modestly. The paired worlds give identical tool observations, so the remaining differences reflect sampling and the small differences between the episodes' histories, such as ids and timings, rather than information about the outcome. The immediate re-issue rate is 16% in committed and 17% in never-executed worlds for `gpt-5.6-luna`, 77% and 90% for `gpt-4.1-mini`, and 70% and 62% for `gpt-5.4-nano`.

Checking first helps, but only as far as Propositions 4 and 6 allow. Without a layer, `gpt-5.6-luna` ends inconsistent in 32% of these episodes, against 70% for `gpt-4.1-mini`, and its failures concentrate exactly where checking cannot work. In look-alike worlds (70% inconsistent) the other user's identical record convinces it that its own write took effect. In crash worlds (66%) the framework replays the pending call after the restart, before the model can intervene. Under heavy-tailed delays (33%) the record is not there yet when the model looks. With exact reads and a short delay, the same model fails in only 3% of the worlds with a lost reply. In the look-alike and late-commit cells the model's observation is insufficient: a more capable model would read the same look-alike and the same empty result, unless it used fields my look-alikes do not distinguish, such as author and creation time, or waited longer than my late commits. Crash replay is different: it is a framework behaviour that no model can influence.

Asking the model to reflect on the error (C7) gave no benefit in hard worlds: it was two to six points worse than the plain agent for every model (Table 11). In lost-reply worlds with exact reads it moved `gpt-4.1-mini` from 80% to 47% and `gpt-5.4-nano` from 40% to 73%, but with fifteen episodes per cell these differences are within sampling noise. Reports were often not honest about the uncertainty. After an ambiguous write, `gpt-4.1-mini` listed an operation as uncertain in 1% of episodes and claimed completion in 96%; `gpt-5.6-luna` listed one in 22% and claimed completion in 80%; `gpt-5.4-nano` listed one in 49%.

Table 10. What a language-model agent does after the first failed or timed-out write when no recovery layer is present (condition C0): its first action, whether it checked state before re-issuing, and what it reported. "Inconsistent" is the share of these episodes whose final state was wrong.

Model	n	Re-issue %	Check %	Wait %	Other %	Checked before re-issue %	Listed as uncertain %	Claimed completed %	Inconsistent %
gpt-4.1-mini	102	80	20	0	0	16.3	1.0	96.1	69.6
gpt-5.6-luna	204	17	69	7	8	72.1	22.1	80.4	32.4
gpt-5.4-nano	102	68	22	0	11	20.2	49.0	64.7	53.9

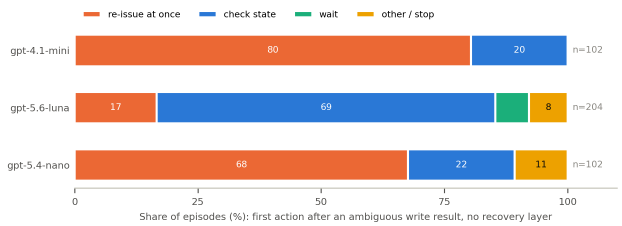


Fig. 6. First action of each model after an ambiguous write result, without a recovery layer (C0).

8.9 Settle under language-model agents (H6)

With Settle underneath, the differences between models disappear (Fig. 7, Table 11). In hard worlds, which are the late-tail, crash and look-alike or lagged-read worlds, each of the three models ends inconsistent in 6.0% of episodes, while every baseline condition of every model lies between 47.8% and 83.6%. Hard and easy worlds as defined before the test runs exclude the shared-aggregate worlds; they hold 8 of Settle's language-model failures, all of them abstentions that Settle reported. Every comparison is significant (largest Holm-adjusted $p < 0.001$), so H6 is confirmed for every model. The three Settle rates are identical because the remaining failures are determined by the world, not by the model. Of 472 ambiguous Settle episodes, 24 ended inconsistent: 16 with a delay beyond the assumed bound and 8 on shared aggregates, the same predicted kinds as with the scripted driver, and none for any other reason. In easy worlds

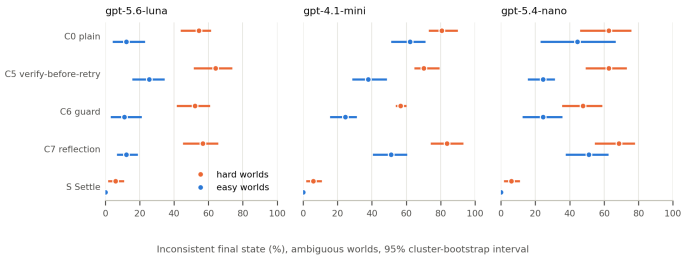


Fig. 7. Inconsistent final states per model and condition in hard worlds (late_tail, crash_after, look-alike or lagged reads) and easy worlds (exact reads; lost_ack, not_exec_err, late). 95% cluster-bootstrap intervals.

no Settle episode of any model ended inconsistent, whereas even the careful `gpt-5.6-luna` loses something there on its own (12.2%).

Identical rates across models are what one should expect when the layer resolves every write before the model sees an error, so they are not by themselves evidence of robustness. A recovery guarantee that depends on how carefully a model reasons would not be a guarantee, and the scripted results already show that Settle's rests on the contract. What the language-model runs add is a check that the guarantee survives the behaviours a model brings and a scripted agent does not: rephrased re-issues, waits and deletions. The main runs bound writes to intents through the harness's step mapping (Section 6); the follow-up repeats them with plan-free binding and gets identical outcomes, and adds a frontier model (Section 9). No agent under Settle removed another user's record, against 6 episodes with such collateral damage under the baseline conditions.

Effect of amendment 2. Before amendment 2, the ledger-key collision had inflated the baselines' rates for the two models that choose their own transfer keys. For `gpt-5.6-luna` without a layer, the hard-world rate moved from 65.7% to 54.5% and the easy-world rate from 37.8% to 12.2%. For `gpt-5.4-nano` the hard-world rate moved from 74.6% to 62.7% and the easy-world rate from 68.9% to 44.4%. H6 was confirmed for both models before and after the amendment, and every number reported here is the corrected one.

Table 11. Language-model agents on the test split: inconsistent final states in hard and easy ambiguous worlds; overclaiming, as a share of all episodes in ambiguous fault modes (all read models, including shared aggregates), where an episode overclaims if its final state is inconsistent although the agent claimed completion and neither the agent nor the layer marked any operation as uncertain or duplicated; turns and cost per episode.

Model	Condition	Inconsistent %, hard worlds	Inconsistent %, easy worlds	Overclaim % (ambiguous)	Turns	US$ / episode
gpt-4.1-mini	C0 plain	80.6	62.2	67.8	8.4	0.0018
gpt-4.1-mini	C5 verify-before-retry	70.1	37.8	55.9	7.1	0.0014
gpt-4.1-mini	C6 guard	56.7	24.4	32.2	8.0	0.0016
gpt-4.1-mini	C7 reflection	83.6	51.1	62.7	7.4	0.0015
gpt-4.1-mini	S Settle	6.0	0.0	3.4	7.2	0.0014
gpt-5.6-luna	C0 plain	54.5	12.2	13.6	9.6	0.0010
gpt-5.6-luna	C5 verify-before-retry	64.2	25.6	47.5	7.8	0.0007
gpt-5.6-luna	C6 guard	52.2	11.1	12.3	9.5	0.0009
gpt-5.6-luna	C7 reflection	56.7	12.2	15.7	9.4	0.0009
gpt-5.6-luna	S Settle	6.0	0.0	3.4	7.6	0.0008
gpt-5.4-nano	C0 plain	62.7	44.4	33.1	5.5	0.0008

Model	Condition	Inconsistent %, hard worlds	Inconsistent %, easy worlds	Overclaim % (ambiguous)	Turns	US$ / episode
gpt-5.4-nano	C5 verify-before-retry	62.7	24.4	42.4	4.8	0.0006
gpt-5.4-nano	C6 guard	47.8	24.4	11.0	5.4	0.0007
gpt-5.4-nano	C7 reflection	68.7	51.1	43.2	5.6	0.0007
gpt-5.4-nano	S Settle	6.0	0.0	3.4	4.8	0.0006

8.10 Cost (H5)

Settle's cost is latency, not model calls. In fault-free worlds it adds 1.01 s per scripted episode relative to the guard (95% upper bound 1.29 s). In ambiguous worlds it adds 15.0 s on average, almost all of it the reconciliation horizon, whose trade-off Section 8.6 traces. Settle *saves* model calls, because the agent rarely sees an error it has to reason about. Its cost per episode relative to the guard is 0.84 for `gpt-5.6-luna` (95% upper bound 0.94), 0.87 for `gpt-4.1-mini` (0.99) and 0.91 for `gpt-5.4-nano` (1.16), against the pre-registered limit of 1.25; the cost criterion is confirmed for all three, and the latency criterion is confirmed.

9. An internally pre-registered follow-up study

The main study left six questions open that only new experiments could settle. Was a simpler tag-based *prevention* policy as good as Settle? Did Settle's implementation gaps (Section 10.1) matter? Would a duplicate that the network delivers late escape it? Do look-alikes from another account defeat content checks as easily as same-account look-alikes do? Does the language-model result depend on the harness's plan mapping? And would a frontier model change it? I answered them in a follow-up study with its own analysis plan, hypotheses F1–F7 and pass criteria, frozen before the first follow-up episode (`docs/07_followup_plan.md`; freeze digest prefix `87ddadb5`). The main study's records and verdicts are unchanged. The follow-up ran 4,272 episodes with no exclusions: 3,468 scripted episodes, 536 episodes with the three original models and 268 with `gpt-5.6-sol`. It cost US$ 6.09 in model calls, under a hard cap of US$ 9.90. All seven hypotheses were confirmed (Table 12). One analysis defect was corrected after the runs: the frozen script treated a rate of exactly zero as missing, so it reported the two F4 criteria on S2 as not confirmed although S2's measured rates (0 of 51 and 0 of 21) met them. Every other verdict is the same under the frozen and the corrected script; both are released (Appendix D). Several follow-up hypotheses check properties that follow from the conditions' definitions, such as that a policy without a final sweep misses redelivery; F1(a), F5 and F7 are the informative ones.

Prevention versus convergence (F1). The new condition Sp is the policy that the corrected Proposition 3 recommends for every write. It keeps the durable journal and the tag, but after an ambiguous outcome it waits out the in-flight bound, counts by tag and re-issues only if no effect exists. It never compensates and has no final sweep. On the H1 population without redelivery and heavy-tailed delays, Sp and Settle were *identical*: neither left any of the 519 paired worlds inconsistent. The two differ only where the theory says they must. On transport redelivery Sp was inconsistent in 42 of 42 worlds and Settle in 0, because preventing one's own re-issues does nothing about a duplicate the transport creates. Under heavy-tailed delays Sp was worse by +9.5 pp (54.8% against 45.2%), but this rests on 4 discordant worlds (exact McNemar $p = 0.125$). In all of them the commit landed between 21.0 and 25.6 s, just past the 20 s bound: Sp's last decision comes Δ after the attempt and Settle's at the end of the episode, a few seconds later (Proposition 7). The difference is therefore a matter of decision timing, which a slightly larger bound for Sp would remove, not a robust advantage of convergence. In exchange, Sp finished the episodes of the H1 population about 3 s sooner on average (paired difference −3.0 s, 95% CI −3.3 to −2.7 s). Over the whole H1 population Sp was inconsistent in 10.8% of worlds against Settle's 3.2%, and 42 of its 65 failures were redeliveries. The comparison settles what Settle's extra machinery buys, and it is narrower than the headline comparison suggests. When the failure is a lost reply, a request that never arrived, a crash, a partial batch or a bounded delay, a tag-based wait-then-count policy is exactly as good. Convergence matters for duplicates that the transport creates.

Settle as designed (F2, F3). The variant S2 closes the first two implementation gaps. It never confirms an ambiguous content-only write by a content match, and its final sweep always waits out the bound. On the 39 ambiguous content-only worlds, S2 left the same share inconsistent as Settle (28.2%). But it overclaimed in none of them, against 6 for Settle, so every content-only failure is now reported. S2's ambiguous content-only writes are reported as uncertain

also when they turned out fine, which is the cost of abstaining. On the H1 population S2 and Settle were again identical. The price of always waiting is paid in every episode: S2 adds 19.8 s to a fault-free episode (95% CI 19.6 to 19.9 s), close to the assumed bound of 20 s. This exceeds the main study's own latency criterion for fault-free episodes (H5: at most 2 s more than the guard), which S2 would therefore fail.

Delayed redelivery (F4). In the new mode `redelivery_late`, the write is acknowledged normally, and the transport delivers an identical copy 8 s later, after a Settle episode without ambiguity has already finished. On the 51 worlds whose write is reversible and attributable, every condition except S2 ended inconsistent in every world, including Settle and the outcome oracle; S2 ended inconsistent in none. On the 21 worlds whose write is an e-mail, every condition ended inconsistent, as Proposition 3 says it must: no client can recall a delivered duplicate. But S2 reported every one of them, while every other condition claimed success in all of them. On the 12 keyed worlds no condition failed, because the service drops the duplicate key. The design choice is therefore real. A sweep that waits only when something looked ambiguous saves about 20 s per clean episode, and misses every duplicate that arrives after the agent has finished. S2 catches only duplicates that arrive within $\Delta + \lambda$ of the last attempt; a redelivery after its horizon is untested and would be missed as well.

Look-alikes from another account (F5). In the new read variant `lookalike_other`, a second, ordinary Gitea user creates the look-alike at the moment of the fault. Content checks without an author filter fared as before: verify-before-retry and the waiting guard were each inconsistent in 64.3% of the 84 worlds. They failed even in some lost-reply and late-commit worlds (6 of 21 each), in a way the main study already showed for same-account look-alikes: after an issue's reply was lost, they resolved the issue's number by its title, could pick the other user's issue, and the task's comments were then missing from the agent's own issue. With an author filter, the waiting guard (Gwa) was inconsistent in 0 worlds, exactly like Settle. Verify-before-retry with the filter (C5a) still failed every late-commit world (100%), because it does not wait. This result limits the scope of Proposition 4 in practice, and it is shown here only for Gitea issues and comments, four fault modes and the scripted driver. The tag is *necessary* when the duplicate-looking record can come from the same account, for instance from another instance of the same agent or another worker using a shared service account, or when the service records no author, as for the mail, rows and stream events of my stack. Where every other writer uses a different account and the service exposes authorship, the client's own account is itself an attribution source in the sense of Definition 1, and an author filter *combined with waiting out the bound* is as good as a tag.

Plan-free intent identity (F6). S−np binds each model write to an intent by the tool and the normalised intent fields of the call itself, without the harness's mapping onto plan steps. For all three original models it produced exactly the same outcome as Settle in every hard and every easy world. In hard worlds each model was again at 6.0%, with no discordant pair. The plan mapping therefore did not contribute to the language-model results. Part of this identity is mechanical: model responses are cached by the request, so where S−np and Settle present a model with the same history it receives the same response, and the two conditions can diverge only after their observations differ. Binding by intent fields does carry the risk that two deliberately identical writes are merged; no task in my suite contains such a pair.

A frontier model (F7). With `gpt-5.6-sol` as the agent and no recovery layer, 52.2% of hard worlds ended inconsistent (35 of 67), close to `gpt-5.6-luna`'s 54.5%. With plan-free Settle underneath, 6.0% did (paired difference −46.3 pp, 95% CI −53.5 pp to −35.4 pp; exact McNemar $p < 0.001$). The frontier model behaved like the careful smaller one. After its first ambiguous write it checked state first in 77% of episodes and re-issued at once in only 16%. Its failures fell in the same cells: 74% of look-alike worlds, 56% of crash worlds and 27% of heavy-tailed-delay worlds were inconsistent, against 0% after a lost reply with exact reads. A more capable model does not escape the cells where the observation is insufficient or the framework re-issues before the model acts. Its cost per episode was US$ 0.0197 with Settle and US$ 0.0223 without. These figures count paid calls only, and the two conditions share cached responses for identical request histories, so they understate both.

Table 12. The pre-registered follow-up study: inconsistent final states unless stated otherwise. Sp: tag wait-then-count; S2: Settle as designed; S−np: Settle with plan-free intent identity; C5a and Gwa: verify-before-retry and guard + wait with an author filter.

Question	Worlds	Result	Verdict
F1(a) Sp vs S, ordinary faults (H1 population)	519	identical: 0 discordant pairs	confirmed
F1(b) Sp vs S, transport redelivery	42	Sp 100%, S 0%	confirmed
F1(c) Sp vs S, heavy-tailed delay	42	Sp 54.8%, S 45.2%	confirmed
F2 S2, content-only writes: overclaim	39	S2 0, S 6	confirmed

Question	Worlds	Result	Verdict
F3 S2 vs S (H1 population); fault-free cost	603	identical; +19.8 s per fault-free episode	confirmed
F4(a) delayed redelivery, reversible writes	51	S2 0%; S, Sp, Gw, C0, O 100%	confirmed
F4(b) delayed redelivery, e-mail: overclaim	21	S2 0%; all others 100%	confirmed
F5 look-alikes from another account (Gitea)	84	C5, Gw 64.3%; C5a 25.0%; Gwa, S, Sp, S2 0%	confirmed
F6 plan-free intent identity, hard worlds	134 / 67 / 67	S−np = S for all three models	confirmed
F7 `gpt-5.6-sol`, hard worlds	67	C0 52.2%, S−np 6.0%	confirmed

10. Discussion

Where idempotency keys and workflow engines fall short. Keys and durable execution answer the question "has this *request* been processed?" The question an agent faces after an ambiguous outcome is a different one: "how many *effects* of this operation exist?" The two coincide only when the service enforces the key, and the census shows that this is the exception outside payments. Without a key on the service, a key the client generates is a string the service ignores, however durable it is, and the effect it should protect can still be duplicated by a late commit, a redelivery or a replayed call. A workflow engine's durable history makes a crashed step resume, but resuming means executing the activity again: the engine provides at-least-once *execution* by design [12], [13], and with key-less tools that becomes at-least-once *effects*. My measurements locate this boundary. In-memory keys and the workflow engine left 60.9% and 61.0% of ambiguous key-less episodes inconsistent, no different from plain retries (61.0%). On keyed writes the engine matched Settle exactly. And on key-less writes its durable history did not stop a crashed step from taking effect twice, in 100.0% of crash-after-dispatch worlds. Even where a key exists, its retention bounds the protection: a key that the service forgets after an hour does not cover an agent that resumes the next day.

What API designers can do. The most useful thing a key-less API can offer an agent is not necessarily a key, but a persisted, queryable, client-controlled field. A metadata map, an external id or a header that the service stores and lets clients look up exactly is enough to build attribution and convergence on the client (Proposition 2). The census shows that many APIs already have such a field and that the gaps are specific. Some APIs persist the field but cannot look it up, such as Pub/Sub attributes or SQS message attributes. Others look it up only through a search documented as lagging by an unstated amount, which weakens assumption (c). Still others return only a count or a sum, which removes attribution entirely (Proposition 5). The common design in which the server returns the new id is no help in the case that matters, since the id is lost together with the reply. Tool annotations such as idempotency hints [21] describe intent but cannot provide either property. A tool description that says how to *find* an effect again would provide it, and the tag can usually live in a field that end users never see: 9 of the taggable key-less APIs in the census document such a field.

When a tag is needed. Proposition 4 is an indistinguishability argument, and the follow-up shows where it binds in practice. A look-alike created by a *different* account was harmless to a content check that also filtered by author and waited out the bound; such a check matched Settle. The tag is needed when the look-alike can come from the same account (another instance of the same agent, or another worker sharing its service account, which is common), and on services that record no author, such as the mail, rows and stream events of my stack. Since one cannot usually rule out the first case, the tag remains the robust choice, but the claim is narrower than "content checks fail under look-alikes"; how often real services lack an author field is something the census did not code.

What agent frameworks can do. Two framework behaviours caused failures in my language-model runs. Replaying a pending tool call after a restart turns every crash during a write into a blind re-issue (Proposition 6). Leaving clean-up to the model is the second: under look-alikes, a careful model removed what it took to be its own duplicate and deleted someone else's record. The model was not careless; by content, the two records were indistinguishable. Both behaviours are fixed by two mechanisms that need no better model: a tag that survives the restart, for instance in a journal written before dispatch, which makes a restart resolve the outcome instead of repeating the call, and attribution, which gates every destructive action.

Prevention, convergence and what each costs. The follow-up study (Section 9) sharpens the choice between the two guarantees. A tag-based wait-then-count policy, which is prevention without a key, was exactly as good as Settle on lost replies, requests that never arrived, held connections, bounded delays, crashes and partial batches, and about 3 s faster per episode of the H1 population. Convergence earned its place for duplicates created by the transport, which no client-side prevention can stop; its small advantage under long delays came from four worlds just past the bound and reflects decision timing rather than a robust difference. A deployment that knows its transport never redelivers and its

delays are bounded can therefore use the simpler policy. One that cannot should converge. If late transport duplicates are possible it should also sweep in every episode, at a cost of about one in-flight bound of latency per episode, because a sweep that runs only after visible trouble misses every duplicate that arrives after the agent has finished; the main study's latency criterion would rule this out, so the choice is a real trade-off.

Convergence is not prevention, and latency is negotiable. Settle allows transient duplicates and removes them by the horizon. That is acceptable for an issue, a row or an event, but not for effects that others react to before compensation, such as a page that wakes someone or a webhook that triggers a shipment. For irreversible effects the theory and my measurements agree that convergence collapses to prevention. Settle then waits out the in-flight bound before re-issuing, and reports an unrecoverable duplicate instead of hiding it. Where even a transient duplicate is harmful, the right tool is a key at the service, and my results quantify what is lost without one. For reversible effects the latency cost is not intrinsic. Proposition 7 shows that the residual risk falls only as the last decision moves later, but nothing requires the agent to wait for that decision: the journal outlives the episode, and a background reconciler can keep counting effects by tag and compensate a late surplus after the agent has reported. My implementation blocks until the horizon and I did not evaluate this asynchronous variant. In my late-tail worlds it could have removed at most the 22 delay failures on reversible writes; the other 14 concern irreversible writes, for which only a longer wait before the re-issue helps.

10.1 Limitations and threats to validity

External validity of the stack. The services are real, but they run on one machine under synthetic load, and the background actor is the only other user. Real deployments have more concurrent writers, but I have no data on how often they produce look-alikes as close as mine. In the main study the background actor uses the same account and writes at the moment of the fault, which is the worst case for checks by content; the follow-up adds look-alikes from a second account. The census establishes how real contracts are distributed over the classes of the theory, but it codes documented behaviour, not behaviour under fault. The two together argue that the classes I test are the ones real services fall into; they do not show how often each fault occurs in production. Production traces of ambiguous outcomes would close this gap, and I know of no public source of them.

Faults. Faults are injected at the network and process level and validated per service against the service's own records (Appendix B). Real outages produce mixtures of faults and delays longer than my twelve-second late commit. The heavy-tailed mode and the replay in Section 8.6 show how the residual risk grows when the in-flight bound is violated, as Propositions 2 and 7 predict, but my delay distribution is a choice, not a measurement. The replay itself is exploratory: it relies on Settle's decision rule, which it verified on 167 of the 168 recorded late-commit worlds with exact reads, and not on new runs at each bound.

Models. I test four OpenAI models at their default settings, all from one provider: three smaller models in the main study and the frontier model `gpt-5.6-sol` in the follow-up, which ran one repeat and two conditions within a fixed budget. The frontier model behaved like the most careful smaller model, checking state first in most episodes and failing in the same cells. Models from other providers, other reasoning settings and agent frameworks that do not replay pending calls remain untested. A model that compares authors and creation times, or waits longer, might do better against less adversarial look-alikes and shorter delays than mine.

Baselines. The guard [8], verify-before-retry [23] and rollback [14] were re-implemented from their published descriptions and may differ from their authors' code in detail. I describe what each re-implementation does (Section 7.3), use the strongest configuration of each family I could build without attribution, and release the code. Settle, the theory and the harness were all designed by me. The frozen predictions, the matched-latency control, the per-mode breakdown and the tag-aware ablations limit how much this can favour Settle, but they do not replace an independent replication. The baselines of the main study are denied tags by design, so part of the headline comparison confirms Proposition 4 rather than testing an engineering choice. The follow-up adds the two fairest competitors I could construct, a tag-based wait-then-count policy and a waiting guard with an author filter, and reports where each matches Settle.

Pre-registration. The freezes are internal: they consist of source hashes recorded in the repository, not of a registration with a third party, and the freeze, runs and writing took place within a few days. The follow-up study has its own plan and freeze, written after review of the main results and before any follow-up episode; its analysis script needed one correction after the runs (Appendix D). Two amendments were made to the main study after its freeze, one of them after the runs. Both are documented with their evidence and re-freezes (Appendix D). The second corrected a defect that had *inflated* the baselines' failure rates; the verdicts of H1–H4 and H6 held before and after it, and H5's cost criterion

for `gpt-5.4-nano` moved from inconclusive to confirmed. The restriction of H6 to hard worlds was decided on pilot data before the freeze, but it is a choice a reader may weigh.

Step mapping and grading. For fault injection and grading, language-model writes are mapped to plan steps by their key fields; a rephrased write that escapes the mapping is treated as a new intent, and its effects are still graded. Grading uses service-side records only. Mail cannot be deleted, so a duplicated e-mail is always counted. For Gitea deletions I rely on a compensation log written by the harness, because deleted issues and comments are no longer visible through Gitea's API.

Tags: visibility and forgery. In my stack the tag of an issue or comment is a visible body line. Real deployments can often place it in a hidden field instead; where only visible fields exist, the tag is a cost in presentation that some teams may not accept. A visible tag can also be copied into another record, for instance when someone quotes the original. Settle would then count the copy as its own, skip a needed write or compensate another user's record. Requiring the tag together with the client's own author account would close most of this gap; my implementation checks the tag only.

What reversal leaves behind. Deleting a surplus issue or row restores the state my grader checks, but creating it may already have sent notifications or triggered webhooks, and another user may have commented on the duplicate before it was removed. The grader does not count notifications, so reversible in my sense means reversible in the service's state, not in its side effects. Interactions are a real hazard: all of rollback's collateral damage arose because it deleted the agent's own issue together with the other user's comment on it.

Implementation gaps. In the main study, three behaviours of the implementation differed from the design that the theory supports: Settle resolved an ambiguous content-only write by content, its final sweep waited only after visible trouble, and in the language-model runs it bound writes to intents through the harness's plan mapping. The follow-up measured the design without these gaps (S2, S−np): it reports every content-only failure, catches delayed redelivery of reversible writes at a cost of about 20 s per episode, and gives identical language-model outcomes without the plan mapping in every ambiguous world. Two gaps remain untested. Compensations ran on a fault-free path, so their own ambiguity was not exercised, and a visible tag could be copied into another record.

Design of the fault grid. The pooled rates depend on the mix of modes I chose. Two modes, redelivery and partial batches, are failed by construction by every condition without a final sweep or row-level checks, and Table 6 therefore also reports the comparison without them. The crash pair penalises only eager re-issuing, because the driver's checkpoint re-issues a step that crashed before dispatch in every condition.

11. Conclusion

An agent that loses the reply to a write does not know whether the action happened, and no amount of reasoning recovers what the observation does not contain. What recovers it is attribution: the ability to count the effects of one's own operation. I defined attributability as identifiability and derived which guarantee each API contract and read model admits. Prevention needs a durable key unless the in-flight bound is known and waited out, and a key protects only within its retention. Convergence needs only attribution, reversibility and an in-flight bound. A policy that must stop at a horizon has to bet between two indistinguishable worlds. And abstention is the honest answer everywhere else. A census of 52 APIs shows that keys are the exception outside payments, while a taggable field is the rule. Settle implements the map with a journal written before dispatch, a tag in a field the API already persists, a reconciliation horizon and compensation gated on evidence. On real services with real faults and under a frozen protocol, Settle converged where the theory says convergence is possible. Its remaining failures were violations of its assumed in-flight bound, whose risk trades against latency, or lay where the theory admits no guarantee. It reported every abstention on a shared aggregate and every unrecoverable duplicate, and in its designed form also every content-only failure and every late-arriving duplicate of an e-mail. An internally pre-registered follow-up located the boundaries of the method more precisely than the main study could. A tag-based prevention policy is as good as convergence on ordinary faults, so convergence is needed mainly for transport duplicates. An author filter combined with waiting is as good as a tag when other writers use other accounts, so the tag is needed for same-account look-alikes and services without authorship. And a frontier model fails without the layer in the same places as smaller ones, while with it all four models converge alike. Keys and workflow engines remain the right tool when the service honours a key. For the many services that do not, the question an agent should ask is not "did my request arrive?" but "how many of my effects exist?"

Declarations

Funding: This research was self-funded by the author and received no external funding.

Competing interests: The author declares no competing interests.

Author contributions: Ankitha Vuyyala: conceptualization, methodology, software, investigation, validation, formal analysis, visualization, and writing (original draft, review and editing). The author reviewed and approved the final manuscript and accepts responsibility for its content.

Data and code availability: The Settle implementation, the baselines, the fault-injection harness and service definitions, the API census, the frozen study protocols and analysis plans, the recorded episode outcomes and the analysis scripts are publicly available at https://github.com/ankitha-vuyyala/action-recovery. The artifacts corresponding to this manuscript are archived in release v1.0 at https://github.com/ankitha-vuyyala/action-recovery/releases/tag/v1.0. The release includes the raw-records archives. They hold the per-episode folders of all 29,394 recorded episodes: the 20,500 episodes of the main study (17,820 scripted and 2,680 language-model episodes) and the 4,272 episodes of the follow-up study, 24,772 in all, together with 4,622 episodes of development, pilot and gate-check runs, which are not part of the confirmatory or follow-up analyses. Each folder contains the configuration, dispatch and checkpoint logs, the grader reports and, for language-model episodes, the full conversation with the model. The archives also hold the response cache of the 17,171 paid model calls, whose model, token usage, cost and time are logged in the repository as well (`runs/cost_ledger.jsonl`). The evaluation records in the repository regenerate every reported number, table and figure, and the response cache replays every model decision without an API key. Third-party materials, including the service container images, remain subject to their respective licenses and attribution requirements.

Corresponding author: Ankitha Vuyyala. Email: ankithavuyyala08@gmail.com

References

[1] E. A. Akkoyunlu, K. Ekanadham, and R. V. Huber, "Some constraints and tradeoffs in the design of network communications," in *Proc. 5th ACM Symp. Operating Systems Principles (SOSP)*, 1975, pp. 67–74, doi: 10.1145/800213.806523.

[2] J. N. Gray, "Notes on data base operating systems," in *Operating Systems: An Advanced Course, ser. Lecture Notes in Computer Science, vol. 60, R. Bayer, R. M. Graham, and G. Seegmüller, Eds. Berlin, Germany: Springer*, 1978, pp. 393–481, doi: 10.1007/3-540-08755-9_9.

[3] P. Helland, "Idempotence is not a medical condition: An essential property for reliable systems," *ACM Queue*, vol. 10, no. 4, pp. 30–46, 2012, doi: 10.1145/2181796.2187821.

[4] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, "Implementing linearizability at large scale and low latency," in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP)*, 2015, pp. 71–86, doi: 10.1145/2815400.2815416.

[5] Stripe, "Idempotent requests," *Stripe API Reference*. [Online]. Available: https://docs.stripe.com/api/idempotent_requests (accessed Oct. 1, 2026).

[6] S. Sun, "Did it happen? Counterfactual evaluation of LLM agent recovery from ambiguous tool outcomes," Research Square preprint, 2026, doi: 10.21203/rs.3.rs-10730245/v1.

[7] S. Gopnal Swamy, *IdempotencyBench: Do LLM agents act exactly once? Measuring idempotency violations under retries*, 2026. [Online]. Available: https://github.com/gssanjana4/idempotencybench

[8] J. Li, "Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents," arXiv:2609.29095, 2026.

[9] A. Trofimov and B. Novikov, "When tool calls succeed but workflows fail: Anomalies at the agent-tool boundary," arXiv:2609.15397, 2026.

[10] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, transactional tool use for reliable agentic workflows," arXiv:2602.14849, 2026.

[11] Z. Chen et al., "Cordon: Semantic transactions for tool-using LLM agents," arXiv:2606.17573, 2026.

[12] Temporal Technologies, "Activity definition," *Temporal Documentation*. [Online]. Available: https://docs.temporal.io/activity-definition (accessed Oct. 1, 2026).

[13] DBOS, "Workflows," *DBOS Docs*. [Online]. Available: https://docs.dbos.dev/python/tutorials/workflow-tutorial (accessed Oct. 1, 2026).

[14] S. Perera, K. Hapuarachchi, F. Leymann, and R. Khalaf, "Robust agent compensation (RAC): Teaching AI agents to compen-

sate," in *Proc. ACM Conf. AI and Agentic Systems (CAIS)*, 2026, pp. 253–262, doi: 10.1145/3786335.3813141.

[15] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 4874–4886, 2025, doi: 10.14778/3750601.3750611.

[16] G. Wang et al., "Consistency and completeness: Rethinking distributed stream processing in Apache Kafka," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, pp. 2602–2613, doi: 10.1145/3448016.3457556.

[17] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 1187–1204.

[18] TigerBeetle, "Reliable transaction submission," *TigerBeetle Docs*. [Online]. Available: https://docs.tigerbeetle.com/coding/reliable-transaction-submission/ (accessed Oct. 1, 2026).

[19] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1987, pp. 249–259, doi: 10.1145/38713.38742.

[20] H. F. Korth, E. Levy, and A. Silberschatz, "A formal approach to recovery by compensating transactions," in *Proc. 16th Int. Conf. Very Large Data Bases (VLDB)*, 1990, pp. 95–106.

[21] Model Context Protocol, "Specification 2025-06-18: Schema reference (ToolAnnotations)," *Model Context Protocol website*. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18/schema (accessed Oct. 1, 2026).

[22] Z. Wang, "Callability is not operability: Controlled interface interventions for LLM agents," arXiv:2608.23628, 2026.

[23] I. K. Mansoor, A. Phadke, and P. Rana, "Verified tool calls improve LLM agent reliability under non-atomic failures," arXiv:2608.02645, 2026.

[24] Z. Sun, X. Wang, and G. Li, "Agentic transaction: Towards ACID-compliant agent systems," arXiv:2608.13900, 2026.

[25] Y. Zhuang, K. Chen, Y. Duan, S. Zheng, J. Li, and X.-Y. Zhang, "AgentRewind: Recoverable execution for long-horizon LLM agents," arXiv:2608.14380, 2026.

[26] K. Yang, P. Li, Z. Wu, K. Xu, H. Huang, and X. Huang, "DART: Semantic recoverability for structured tool agents," arXiv:2605.23311, 2026.

[27] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," arXiv:2603.20625, 2026.

[28] J. Xu, L. Fan, Z. Wang, X. Li, and H. Liu, "Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions," arXiv:2608.01710, 2026.

[29] A. Gupta, "ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions," arXiv:2601.06112, 2026.

[30] A. Sigdel and R. Baral, "ToolMisuseBench: An offline deterministic benchmark for tool misuse and recovery in agentic systems," arXiv:2604.01508, 2026.

[31] D. Zhu et al., "When tools fail: Benchmarking dynamic replanning and anomaly recovery in LLM agents," arXiv:2606.05806, 2026.

[32] G. Tan et al., "AgentChaos: Chaos engineering for agent systems via programmatic fault injection," arXiv:2608.06790, 2026, to appear at ASE 2026.

[33] Z. Zheng, J. Long, B. Li, and J. Yao, "Stale does not mean unsafe: Guard precision for tool-using LLM agents under infrastructure state races," arXiv:2609.29522, 2026.

[34] Z. Zhai, M. Li, and X. Wang, "Revisable by design: A theory of streaming LLM agent execution," arXiv:2604.23283, 2026.

[35] L. Advani, "From confident closing to silent failure: Characterizing false success in LLM agents," arXiv:2606.09863, 2026.

[36] The Gitea Authors, *Gitea: Git with a cup of tea*, 2026. [Online]. Available: https://github.com/go-gitea/gitea

[37] Mailpit contributors, *Mailpit: An email and SMTP testing tool and API for developers*, 2026. [Online]. Available: https://github.com/axllent/mailpit

[38] Redis, "Redis Streams," *Redis Documentation*. [Online]. Available: https://redis.io/docs/latest/develop/data-types/streams/ (accessed Oct. 1, 2026).

[39] SeaweedFS contributors, *SeaweedFS: Distributed storage system for object storage (S3), file systems, and Iceberg tables*, 2026. [Online]. Available: https://github.com/seaweedfs/seaweedfs

[40] Shopify, *Toxiproxy: A TCP proxy to simulate network and system conditions for chaos and resiliency testing*, 2026. [Online]. Available: https://github.com/Shopify/toxiproxy

[41] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement

learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 8634–8652.

[42] OpenAI, "Create a model response," *OpenAI API Reference*. [Online]. Available: https://developers.openai.com/api/reference/resources/responses/methods/create (accessed Oct. 1, 2026).

Appendix A. Proofs

Throughout, a policy is a function from the client's history to its next action, worlds are drawn from a prior, and $N_\ell(t)$ is the number of standing effects of operation ℓ at time t.

Proposition 1. *(i) Prevention with a durable key.* Let κ be a function of durable state, so that every attempt of ℓ, before and after any crash, carries the same κ. Let e_1 be the time of the first execution of any attempt. By the key contract, the service records κ at e_1, and every execution with key κ in $[e_1, e_1 + r)$ creates no new effect. If every execution of every attempt falls in that interval, exactly one effect is created. Re-issuing until an acknowledgement or a refusal arrives ends with that effect acknowledged, provided each attempt is delivered with positive probability. With a known bound Δ, every execution falls in $[t_1, t_{\text{last}} + \Delta]$, so $t_{\text{last}} - t_1 + \Delta < r$ suffices. Because the loop has no time bound, prevention holds with the probability that the loop finishes before this condition fails, and with certainty only if the number of attempts is capped.

(i) Retention. An execution at $e > e_1 + r$ is treated as new. In a world where a_1 executed at e_1 and its acknowledgement was lost, a late execution after $e_1 + r$, whether of a re-issue or of a delayed original, creates a second effect. After that point the client must resolve the outcome by another attribution source before re-issuing, which is the situation of Propositions 2 and 3. State-based expiry is the same argument with the state change in place of $e_1 + r$.

(ii) Failure with an in-memory key. Let the client crash after dispatching attempt a_1 with key κ_1 and before recording it. After the restart a fresh key $\kappa_2 \neq \kappa_1$ is drawn. In a world in which a_1 is delayed and both a_1 and the re-issued a_2 execute, the service sees two distinct keys and creates two effects.

(iii) Without a key. This is Li's Proposition 1 [8], restated in my notation. Take two worlds that agree up to time t: in w_1, attempt a_1 never executes; in w_2, it executes at $t + \epsilon$. They are observation-equivalent up to t. A policy that re-issues at or before t duplicates in w_2, and a policy that never re-issues leaves ℓ missing in w_1. If Δ is unknown, such a t exists for every waiting time.

Proposition 2. Consider one round, and let $t^\star = t_{\text{last}} + \Delta + \lambda$.

1. By (d), every attempt dispatched up to t_{last} has executed by $t_{\text{last}} + \Delta$ or never will.
2. By (b) and (c), a read at $t^\star$ returns exactly the standing effects that carry τ and executed before $t^\star - \lambda$, which by step 1 are all that will ever exist from these attempts.
3. By (f) and (g), the effects that carry τ are exactly the standing effects of ℓ, and no other actor removes them. The count read at $t^\star$ therefore equals the eventual count of the attempts made so far.

If $N = 0$, no attempt executed and none is pending, so a re-issue under the same τ cannot race an earlier attempt, and a new round begins. If $N > 1$, targeted compensation of $N - 1$ effects chosen by id leaves exactly one, using (e). By (e) a compensation may be retried through the faulty channel until a further read, after another lag λ, confirms $N = 1$. If $N = 1$, the policy stops. By (a), τ survives crashes, so any history resumes the same procedure without creating an attempt it cannot attribute. Each round re-issues at most once, and a round ends without a re-issue with probability at least the probability that its attempt executes, so the number of rounds is finite with probability one if that probability is positive. No effect of another actor is touched, because only effects that carry τ are compensated and, by (g), they all belong to ℓ.

For the bound, drop (d). A round that ends at horizon H can leave an inconsistent state only if some attempt executes after $H - \lambda$: otherwise steps 2 and 3 hold at H. For attempt i, dispatched at t_i, this requires that it executes at all and that $\delta_i > H - \lambda - t_i$, which has probability $p_i(1 - F(H - \lambda - t_i))$ if delays are independent of whether the attempt executes. A union bound over the attempts gives the stated bound.

Proposition 3. If effects cannot be neutralised, $N_\ell = 1$ at quiescence implies that exactly one execution happened. That is prevention, so Proposition 1(iii) applies whenever Δ is unknown. If Δ is known, waiting until $t_{\text{last}} + \Delta + \lambda$ before the count makes $N = 0$ equivalent to "no attempt executed or pending" (Proposition 2, steps 1–3), so a re-issue in that case cannot create a second execution. A transport redelivery is an execution that no client action caused and that no client action can prevent: in a world with redelivery and one without, the client's history up to the first execution is the same, and without a key the service accepts both executions. With a tag, the client can detect the second effect and report it.

Proposition 4. In w_1 the client's attempt executed and created effect e. In w_2 it did not, and another actor created e' with the same content. A content read returns one matching effect in both worlds, and every other observation is identical, so the two worlds are observation-equivalent while $N_\ell = 1$ in w_1 and $N_\ell = 0$ in w_2. A content-only policy takes the same action in both worlds. If it skips, ℓ is missing in w_2. If it re-issues, ℓ is duplicated in w_1. If it re-issues and then removes one of the two matching effects, it either removes its new effect, which leaves ℓ missing in w_2, or removes the earlier match, which in w_2 is e' (collateral). Either way it is wrong in w_2. With a tag, e carries τ and e' does not, so a read by tag distinguishes the two worlds.

Proposition 5. *If.* With exclusive writing, the aggregate changes only through the client's own effects, so $N_\ell = (A(t) - A(t_0))/u$, where u is the unit of the operation, A the aggregate and $A(t_0)$ its value read before the first attempt; ℓ is attributable. *Worst case.* Suppose another actor may add one unit of the same size during the window. Then a world in which the client's attempt executed and the other actor's did not, and the reverse world, give the same aggregate. They are observation-equivalent under an aggregate read, while N_ℓ differs, so any decision about re-issuing or compensating is wrong in one of them. A fungible compensation removes "a unit" and cannot tell whose.

Proposition 6. Let the client crash after dispatching a_1 and before recording its outcome. After the restart, compare a world in which the crash happened just before the dispatch, so that ℓ never ran, with one in which it happened just after, so that ℓ may have executed. Without a durable trace of the dispatch and without a recoverable tag, the persisted state is the same in both. A policy that issues ℓ duplicates it in the second world if a_1 executed. A policy that does not issue leaves it missing in the first. A policy that treats ℓ as possibly dispatched and resolves it by reading needs a tag it can count by, and an in-memory tag is lost. Conversely, if τ is recoverable, either from a write-ahead record or deterministically from durable step identity, the policy "treat the step as possibly dispatched, wait $\Delta + \lambda$, count by τ, issue if $N = 0$" is correct in both worlds by Proposition 2.

Proposition 7. Fix a history h up to t_d that satisfies the premise. Because ℓ is attributable, the last read is exact for effects that executed before $t_d - \lambda$. Among the worlds consistent with h, the pending attempt either never executes (w_0) or executes after $t_d - \lambda$ (w_L); the policy cannot tell them apart. After t_d it takes no action on ℓ, so the number $k \in \{0, 1\}$ of its other effects of ℓ that remain standing is fixed at t_d. Here $k = 0$ means that it did not re-issue or that it compensated its re-issue, and $k = 1$ means that it re-issued or kept its re-issue. The final count is k in w_0 and $k + 1$ in w_L, so $k = 0$ is wrong in w_0 and $k = 1$ is wrong in w_L. A policy that chooses $k = 1$ with probability q is inconsistent with probability

$$\begin{aligned} & q \Pr[w_L \mid h] + (1 - q) \Pr[w_0 \mid h] \\ & \quad \geq \min\{\Pr[w_0 \mid h], \Pr[w_L \mid h]\}. \end{aligned}$$

Settle chooses $k = 1$, so its error at h is $\Pr[w_L \mid h]$, and it attains the minimum if and only if $\Pr[w_L \mid h] \leq \Pr[w_0 \mid h]$. If Settle's own re-issue may fail to execute, its error is larger by $\Pr[w_0 \mid h]$ times the probability of that failure.

Appendix B. Environment validation

Before the test runs, and on the frozen code, the validation ran 510 trials in total, ten per service and fault mode. The hidden outcome was read from the services' own records (validation run `20261001-164459`). Table 13 reports how many trials produced the intended outcome; 493 of 510 did.

Table 13. Environment validation: trials whose realised outcome (number of executions, read from the service) matched the intended hidden outcome of the mode. The SeaweedFS conditional create turns a redelivered request into a refusal (one execution, as intended).

Service	none	lost ack	not exec err	held	not exec to	late	redelivery	partial
Gitea issue	10/10	10/10	10/10	10/10	10/10	10/10	10/10	
Mailpit SMTP	10/10	10/10	10/10	10/10	10/10	10/10	10/10	
PostgreSQL row	10/10	10/10	10/10	10/10	10/10	10/10	10/10	
PostgreSQL batch	10/10							10/10
Redis stream	10/10	10/10	10/10	10/10	10/10	10/10	10/10	
Redis counter	10/10	10/10	10/10	10/10	10/10	10/10	10/10	
SeaweedFS S3	10/10	10/10	10/10	10/10	10/10	10/10	10/10	
TigerBeetle	10/10	10/10	0/10	10/10	3/10	10/10	10/10	

Three further checks passed:

- **Late commits.** In every late-commit trial, the write landed after the caller had given up (minimum share 100%).
- **Tag fidelity.** The operation tag was read back byte-exact on every taggable service (minimum 100%).
- **Observation equivalence.** For every service, the paired modes (lost_ack vs not_exec_err; held vs not_exec_to vs late) produced identical tool-level observations: the total variation distance between the distributions of (outcome kind, exception type) was 0. Elapsed times differ by tens of milliseconds at most, well below the variation between services.

For one service the raw client exception *did* differ between the paired modes: the SMTP client raised different exception types for a reply cut after commit and a connection cut before delivery. The tool layer therefore normalises all transport failures to one exception and message, while the raw exception is kept in the logs for audit. A deployed agent whose tool layer passes raw exceptions through may be able to tell some of these worlds apart. My results are thus conservative for such agents only in that respect.

The TigerBeetle exception. The TigerBeetle client library retries internally and deduplicates by session. A request whose first transmission was cut is therefore often delivered by the library's own retry: the not_exec modes realised the intended *absent* outcome in 0 and 3 of 10 trials. This is the native-key contract working as designed. I keep these cells as measured, and they enter no equivalence claim.

Crash modes are validated per episode by the child process's exit code (75 at the crash, 0 after the restart). The late-tail mode uses the same mechanism as the late mode, with a per-world delay.

Appendix C. Agent interface

System instruction (all language-model conditions):

> You are an autonomous operations agent. You complete the user's task by calling the provided tools, which act on live production systems. The user is not available while you work, so do not ask them questions. Follow the task instructions exactly. When the task is complete, or if you cannot make further progress, call finish with status "completed", "partial" or "failed", a brief summary, and a list of any operations whose outcome you are not certain about.

User message. The task goal, followed by the sentence "Each action must take effect exactly once." An example for T7:

> Triage ticket TCK-5476 for Woodgrove: open the issue 'Ticket TCK-5476: Woodgrove cannot export reports' (body: 'Reported by Woodgrove. Priority 3.'), add exactly these two comments to it, in order: 'Triage: reproduced for Woodgrove; assigning to reports team.' and 'ETA for Woodgrove: fix in next release.', and record triage:TCK-5476 with amount 3. Each action must take effect exactly once.

Reflection prompt (condition C7, sent after any turn with a tool error):

> One or more of your tool calls returned an error. Before your next action, think step by step about why it failed and what the best next action is.

Nudge (sent if a turn contains no tool call; at most three times):

> Continue the task using the tools. When you are done, call finish.

Tools. Each task exposes the following tools. Their descriptions state documented service behaviour only; they say nothing about faults or recovery methods.

- *Writes:* create_issue, add_comment, send_email, send_bulk_email, insert_record, insert_records, append_event, adjust_stock, increment_counter, create_transfer (with an optional transfer_id, described as an idempotency key), put_object.
- *Reads:* list_issues, get_issue, search_issues (described as updated asynchronously), search_sent_email, find_records, list_events, get_stock, get_counter, find_transfers, object_exists.
- *The agent's own corrections:* delete_issue, delete_comment, delete_record, delete_event, reverse_transfer.
- *Control:* wait (up to 60 s) and finish.

A task sees only the writes it needs, their reads and corrections, wait and finish.

Limits. At most 30 model turns and 40 tool calls per episode, with one model call per turn. Parallel tool calls were disabled, and `max_output_tokens` was 6000, including reasoning.

Endpoint and settings.

- *Endpoint and reasoning:* the OpenAI Responses API, stateless (`store=false`), with each model at its default reasoning setting.
- *Carried reasoning:* for reasoning models, the encrypted reasoning items are passed back with the transcript.
- *Model versions:* the resolved versions are recorded per call in the cost ledger.
- *Caching:* responses are cached by the SHA-256 of the request, which covers the model, instructions, input items, tools and parameters, together with the repeat index.

Crash and resume. The conversation is checkpointed to disk after every model turn and tool result. After a crash, the restarted agent re-executes every function call that has no recorded output. This is the replay semantics of common agent frameworks, applied identically in all conditions.

Step mapping. Faults target a plan step. A model's write is mapped to a plan step when its key fields match the step after normalising case, white space and trailing periods: the title for issues, the recipient and subject for mail, the item for records, the memo for transfers. For comments, the mapping uses the issue and the text, and falls back to the next unused comment step of that issue. A write that matches no step is an extra intent, and the grader still sees its effects.

Appendix D. Pre-registration and deviations

Freeze. The protocol was frozen on 2026-10-01 22:37:09 UTC. The freeze consisted of SHA-256 hashes of 43 files, with digest prefix `51df7cbdbebd`. The frozen files were:

- all harness, recovery-layer, driver, task, grader and analysis code;
- the container definitions;
- the study design;
- the theory;
- the analysis plan;
- this protocol.

The test presets check the freeze before every start or resume, and each start is logged with the digest.

Order of work.

1. Environment validation and a scripted development grid on the development split.
2. A paid pilot of 84 recorded episodes on the development split: four models (gpt-5.6-luna, gpt-5.4-mini, gpt-5.4-nano and gpt-4.1-mini), five conditions, easy worlds only. gpt-5.4-mini was not carried into the main study: in the pilot it cost about four times as much per episode as gpt-5.6-luna or gpt-5.4-nano (US$ 0.0043 against 0.0011) and, like gpt-5.4-nano and gpt-4.1-mini, re-issued writes without checking, so it would have added cost without adding a distinct behaviour.
3. A check of the hard worlds with one model.
4. Full re-validation on the final code.
5. A harness check of the three test-only templates on a non-test instance; only errors and fault-free outcomes were inspected.
6. The freeze.
7. The test runs.
8. The analysis.

Hypotheses (exact criteria; all evaluated by `analysis/confirmatory.py`):

- **H1.** In class-A, targeted-reversible, ambiguous worlds, Settle's inconsistency is at most 5%. Settle is also at least 15 points below the best non-oracle baseline, with a 95% cluster-bootstrap upper bound below 0 and a Holm-adjusted exact McNemar $p < 0.05$. The best baseline is chosen on test data, which is conservative.
- **H2.** Under look-alikes, each content-based method (C5, C6, S–t) is inconsistent in at least 10% of ambiguous worlds, and Settle's rate is at most half of each, with a CI upper bound below 0 and Holm $p < 0.05$.
- **H3.** On class-K writes, Settle is equivalent to the workflow engine with durable keys (C4): the 90% CI of the difference lies within ±5 points.
- **H4.**
  (a) With a shared aggregate, every non-oracle condition is inconsistent in at least 25% of ambiguous worlds.
  (b) Settle overclaims in at most 5% of shared-aggregate or irreversible ambiguous worlds.

(c) In irreversible worlds, every condition shows duplicates or missing effects.

- **H5.**
  - Settle adds at most 2 s in fault-free worlds (95% upper bound) and at most $\Delta + 10$ s on average in ambiguous worlds, both relative to the guard.
  - Per model, Settle's US$ per episode relative to the guard is at most 1.25 (95% upper bound).
- **H6.** For every model, in hard worlds (late_tail, crash_after, look-alike or lagged reads), Settle is below each language-model baseline, with a CI upper bound below 0 and Holm $p < 0.05$.
- **Ablations.** Each ablation is at least 10 points worse than Settle in the cell where its component is needed (lower 95% bound above 0).

Deviations from the original study design. Each of the following was decided on development data before the freeze, and is logged with its evidence in the released change log.

1. *H1, statistical model.* The mixed-effects logistic model was replaced by cluster bootstrap plus exact McNemar, because of quasi-separation at near-zero rates.
2. *H3, comparator and margin.* The comparator changed from in-memory keys (C3) to durable keys (C4), and the margin widened from ±3 to ±5 points. The reason is that in-memory keys fail in crash worlds exactly as Proposition 1 predicts, and that contrast is reported separately.
3. *Hard worlds in the language-model grid.* The pilot showed that a reasoning model checks state before re-issuing. The language-model grid therefore gained the hard worlds, and H6 was restricted to them.
4. *Settle guards the agent's own deletions.* This was added after a development episode in which the agent deleted another user's look-alike issue.
5. *The OpenAI endpoint changed* from chat completions to the Responses API. Chat completions refused function tools for gpt-5.6-luna, the main model of the language-model study (the only one that reasons by default, and the one run with two repeats), at its default reasoning setting, before any charge.

Amendments after the freeze. Both amendments are recorded in the protocol's amendment log with their evidence. Each was followed by a re-freeze, and every freeze is kept.

- **Amendment 1 (during the runs; re-freeze 2026-10-02 00:10:16 UTC, digest prefix `a674ca5aedfc`).**
  - *Cause.* The spend guard stopped the first model run on a Windows race in the lock file of the cost ledger.
  - *Fix.* The lock treats `PermissionError` like `FileExistsError`.
  - *Impact.* The lock guards only the spending ledger, so no outcome could change. The stopped episode was re-run.
- **Amendment 2 (after the runs; re-freeze 2026-10-03 05:06:59 UTC, digest prefix `dd8b4adba131`).**
  - *Cause.* The audit before writing found that models choosing their own transfer keys collided with the same key chosen in other episodes that replay the same task. TigerBeetle ids are global, and the tool layer reported the ledger's refusal as success.
  - *Fix.* Keys are now scoped to the episode, and refusals are surfaced as errors.
  - *Re-run.* Every T2 and T8 language-model episode, in all conditions, was re-run (540 episodes). The 12 episodes whose agent process had crashed without a report (9 language-model, 3 scripted) were also re-run; one of them was already among the T2/T8 episodes. A crashed agent process now counts as unfinished.
  - *Scope.* The scripted grid is unaffected by the key collision. Its keys are unique per run, and a status scan of its ledger calls found no `EXISTS_WITH_DIFFERENT_*` refusal; the only repeat answers were the expected `EXISTS` for re-issued keys. Its only re-runs were the three crashed episodes.
  - *Reporting.* Section 8.9 gives the numbers before and after. The verdicts of H1–H4 and H6 did not change. H5's cost criterion for `gpt-5.4-nano` moved from inconclusive to confirmed.

Exploratory analyses added after the runs. Two analyses were added while writing, after all test runs and the confirmatory analysis had finished: the API census (Section 5) and the horizon replay (Section 8.6). Neither enters any hypothesis, neither changes a confirmatory number, and both are labelled as exploratory where they appear. The census was coded from public documentation and does not use the test data. The replay reads the recorded test episodes, but computes no rate that a hypothesis depends on. Their code lives outside the frozen source tree, so the freeze still verifies.

Follow-up study. The follow-up study (Section 9) has its own analysis plan (`docs/07_followup_plan.md`) and freeze (digest prefix `87ddadb5`), written after the main results had been reviewed and before any follow-up episode ran; the main study's freeze was archived first. Its spending was capped at US$ 9.90 and came to US$ 6.09. A free smoke

check on the development instance confirmed that the new fault mode and the new read variant realised their intended hidden outcomes before the grids ran. One correction was made after the runs, to the analysis script only: in two F4 criteria the frozen code read a missing rate as `rate or 1`, which also turns a measured rate of exactly 0 into 1, so the criteria "S2 ≤ 5%" and "S2 overclaims ≤ 5%" failed although S2's measured rates were 0%. With the correction both criteria are confirmed; every other result of the script is unchanged. The frozen script is released unchanged at `analysis/followup.py`, so the freeze check still passes, and the corrected copy, which also treats a missing rate as failing in every criterion, is released at `paper/followup/followup_corrected.py` with both outputs. Under the frozen script, F4(a) and F4(b) read "not confirmed".

Exclusions. Unfinished episodes (runner errors, grading errors, model errors or budget stops) are archived and re-run. No scripted or language-model record remained unfinished, so none is excluded.

Appendix E. The API census

Table 14 lists the 52 coded APIs. "Key" records whether the examined create operation has a server-side key, and its documented retention. "partial" marks the near-keys discussed in Section 5, which I do not count as keys. "Lookup by tag" is the best documented way to find an object by the value of the tag field: an exact filter or query (*exact*), a search documented as lagged or fuzzy (*search*), only scanning a list of recent objects (*list scan*), or *none*. "Undo" records whether the effect can be undone through the API. The released table also holds whether the tag field is hidden from end users, the coder's notes, and a link to every documentation page relied on. Documentation was read in October 2026, and vendors change their APIs, so individual entries may have changed since.

Table 14. The census: write contracts of 52 APIs, coded from vendor documentation. Class K: server-side key; A: tag field with exact lookup; A−: tag field with only lagged search or list scanning; C: no usable tag field or no lookup.

API (operation)	Class	Server-side key (retention)	Tag field	Hidden	Lookup by tag	Undo
Stripe (create PaymentIntent)	K	native (24 h)	metadata	yes	search (<1 min lag)	yes
PayPal Orders v2 (create order)	K	native (6 h)	purchase_units.custom_id	yes	list scan	partial
Adyen Checkout (/payments)	K	native (7–14 d)	reference / metadata	unclear	none	yes
Square (CreatePayment)	K	native (undocumented)	reference_id	unclear	list scan	yes
Braintree (transaction sale)	A	partial (30 s duplicate check)	order_id	unclear	exact	yes
Shopify Admin (orderCreate)	A	none	metafields	yes	exact	yes
WooCommerce (create order)	A−	none	meta_data	unclear	list scan	yes
Razorpay (create order)	A	partial (unique receipt rejected)	receipt	unclear	exact	no
Wise (create transfer)	K	native (undocumented)	customerTransactionId	no	list scan	partial
Checkout.com (request payment)	K	native (72 h)	reference / metadata	unclear	exact	yes
Mollie (create payment)	K	native (1 h)	metadata	unclear	list scan	yes
GoCardless (create payment)	K	native (≥ 30 d)	metadata	unclear	list scan	partial
GitHub (create issue)	A	none	label (exact) / body (search only)	no	exact	partial
GitLab (create issue)	A	none	labels	no	exact	yes
Jira Cloud (create issue)	A−	none	issue property / labels	yes	search (index lag)	yes
Linear (issueCreate)	A	partial (client-chosen id)	id (UUID)	unclear	exact	yes
Asana (create task)	A	none	external.gid	yes	exact	yes
Trello (create card)	A−	none	desc	no	search	yes
Notion (create page in database)	A	none	text property	no	exact	partial
Confluence Cloud (create page)	A	partial (unique title per space)	title	no	exact	yes
Bitbucket Cloud (create PR comment)	A	none	content.raw	no	exact	yes
Azure DevOps (create work item)	A	none	hidden custom field	yes	exact	yes
monday.com (create_item)	A	none	text column	no	exact	yes
ClickUp (create task)	A	none	text custom field	no	exact	yes
Slack (chat.postMessage)	A−	none	metadata	unclear	list scan	yes
Discord (create message)	A−	partial (nonce for minutes)	nonce	unclear	list scan	yes
Microsoft Graph mail (sendMail)	A	none	extended property	unclear	exact	no
Microsoft Teams (Graph) (channel message)	C	none	none	n/a	list scan	partial
Gmail (messages.send)	A−	none	custom header	yes	search	no
SendGrid (mail send)	A	none	custom_args	yes	exact (paid add-on)	no
Mailgun (send message)	A	none	o:tag	unclear	exact (events)	no
Twilio SMS (create message)	C	none	none	n/a	list scan	no
Postmark (send email)	A	none	Metadata	yes	exact	no

API (operation)	Class	Server-side key (retention)	Tag field	Hidden	Lookup by tag	Undo
Amazon SES v2 (SendEmail)	A–	none	EmailTags	yes	list scan (sampled)	no
Telegram Bot (sendMessage)	C	none	none	n/a	none	partial
WhatsApp Cloud (send message)	C	none	biz_opaque_callback_data (webhook only)	yes	none	no
Intercom (create message)	C	none	none	n/a	search (content)	partial
Salesforce (create record)	A	partial (upsert by external id)	External ID field	unclear	exact	yes
HubSpot (create contact)	A	partial (unique property)	unique property	unclear	exact	yes
Zendesk (create ticket)	K	native (2 h)	external_id	unclear	exact	yes
Freshdesk (create ticket)	A–	none	custom field / tag	unclear	search (minutes lag)	yes
Google Calendar (events.insert)	A	partial (client id; collisions not guaranteed)	extendedProperties.private / id	unclear	exact	yes
Microsoft Graph calendar (create event)	K	native (undocumented)	extended property	unclear	exact	partial
Airtable (create records)	A	partial (upsert on merge fields)	text field	no	exact	yes
Google Sheets (values.append)	A–	none	cell value	no	list scan	partial
Amazon S3 (PutObject (If-None-Match))	K	conditional create (no expiry)	object key	yes	exact	yes
Amazon DynamoDB (PutItem (attribute_not_exists))	K	conditional create (no expiry)	primary key	yes	exact	yes
Google Cloud Pub/Sub (publish)	C	none	attributes (no lookup)	yes	none	no
Apache Kafka (producer send)	A–	partial (producer session only)	record header	yes	list scan	no
Elasticsearch (index (op_type=create))	K	conditional create (no expiry)	_id	yes	exact	yes
PagerDuty Events v2 (trigger)	K	native (while alert open)	dedup_key	unclear	exact	partial
Amazon SQS (SendMessage)	C	partial (FIFO 5 min)	attributes (no lookup)	yes	none	partial